Context-Aware Agentic Orchestration for Adaptive Edge–Cloud AI Inference Using Reinforcement Learning

Name: Najish Mahmud

Student ID: 2514376

MSc Project Report

Unit Title: MSC Project - Artificial Intelligence

Unit Coordinator: Dr Renxi Qiu

Contents

1 Abstract

Existing cascading systems, namely BranchyNet, MSDNet, EdgeBERT and RouteLLM, route requests using fixed confidence thresholds or hardware timers. They do not consider live infrastructure states, such as cloud queue depth or network delay. The objective of this project is to build a context aware agentic gateway. This solution consists of an edge context protocol, a dynamic Medallion pipeline, MCP skills, intelligent PPO agents, a bidirectional feedback loop, and a policy orchestration layer. All these components help make routing decisions in real time. The PPO agent is a trained frozen policy. It was produced in an offline Gymnasium simulation with domain randomization. The policy is then used in the agentic gateway to make routing decisions. A static threshold baseline system escalates to cloud on every request under bursty and steady traffic. This proposed gateway system escalates far less, 0.5% in bursty traffic and 1% in steady traffic. It also removes SLA violations entirely under bursty load, against 59.5% to 60.5% for the static thresholds. The system was tested against five core baselines and eight baselines drawn from the literature in three traffic scenarios and a seven-run ablation study. Another ablation run compares a single shared frozen policy against a policy orchestration layer that can reuse, fine-tune or train a new policy for each client or scenario. This policy orchestration layer is also configured to use LLMs as an oversight mechanism which can escalate its own decision when necessary.

2 Introduction

2.1 Problem Statement

A confidence-only cascade router like RouteLLM (Ong et al., 2024) reads a request and decides whether a cheap or expensive model should answer it. Its router is trained on human preference data, so the only thing it reads at request time is the request itself. When an edge device answers that request, it is under continuous CPU and thermal stress. This kind of stress can allow the present-day neural networks to produce wrong results (Guo et al., 2017). Even though the softmax output shows the model is confident, the prediction has already degraded. A router that only considers confidence cannot identify this specific issue and thus cannot differentiate an easy input from a hard one. Thus the system returns a confidently wrong answer.

This is the first failure mode, Failure A, where the routing decision never sees the operational health of the edge device at the moment of the inference. BranchyNet (Teerapittayanon et al., 2016) and MSDNet (Huang et al., 2018) also share the same problem. They both rely on a confidence or entropy threshold or a computed budget fixed at the design time. Neither of the systems reads the real-time condition of the infrastructure. EdgeBERT (Tambe et al., 2021) does react to the state of the hardware. But it only looks at its own chip's entropy and voltage-frequency envelope.

A second failure mode, Failure B, sits on top of the first. Present day cascade systems can run the same fixed enrichment pipeline over every request. But no outcome ever travels back to it. The system does not know whether the SLA was met or not, or whether the cloud could have handled the request better. So the next request is handled in the same way for similar conditions. They have no pipeline where the decisions are made differently based on the edge's current state, and no return path where the outcome of the decision is reported back to the router.

This whole scenario motivates the research question behind this project. What is gained by replacing static, passive threshold based routing with a genuinely active orchestration system that can see live data about its environment and reason over it to take actions and in the end, weigh the outcomes of those actions?

2.2 Aims and Objectives

1. Survey the landscape of cascade systems, cloud-edge offloading and agentic AI architectures from 2016 to 2026. Compare the artifact against RouteLLM, the NSGA-II router, Splitwise, GreenServ and PROTEUS.
2. Model the escalation decisions as a Markov Decision Process. Here, the state is a fixed 13 slot vector which includes per edge resource stress. Its conditional slots are masked out when the dynamic Medallion pipeline does not populate them. The actions are *route_to_edge*, *escalate_to_cloud*, or *fetch_extended_context*.
3. Implement the dynamic Medallion pipeline, register the metrics in a bronze registry, enrich them conditionally into silver and produce a masked gold state vector.
4. Carry out the Edge Context Protocol through self-reports from edge and backward loops.

5. Use an offline domain randomized Gymnasium simulation to train PPO policies. The trained frozen policy is then deployed as its own services which is accessible via HTTP endpoints.
6. Provide a dynamic registration and *list_tools()* method of the MCP skill interface.
7. Performance test against five core baselines and eight literature baselines, evaluate on three different traffic conditions, conduct a multi-run ablation study and live test in multi region cloud environment.

2.3 Contributions

1. **Edge as an actor:** The Edge Context Protocol is a new way of designing edge devices. Instead of just returning predictions, the device actively reports its own calibration state. This closes Failure A, a whole class of error that BranchyNet, MSDNet, EdgeBERT and RouteLLM cannot even detect.
2. **Medallion Pipeline with Dynamic Components**: A pipeline that changes what it measures per request, driven by data from the edge. Which slots of the state vector carry a value varies, while the vector's own dimension stays fixed at 13.
3. **SLA remaining and predicted RTT as per-request signals at routing level**: Every routing decision sees the current request's own SLA remaining and predicted network RTT, not an aggregate cluster-level budget. That granularity is missing from Splitwise (cluster/phase-level scheduling) and every other cascade baseline studied here. It is a soft, learned preference. The reward function penalizes an SLA violation directly, but nothing yet blocks a doomed escalation before it happens. Section 5 states this distinction plainly. It also names a Silver-layer signal that was computed for that harder guarantee, but is not yet wired into any decision path.
4. **MCP Skills Interface at Inference Latency**: A dynamic per-request tool discovery and registration pattern that lets a synchronous ML service make use of infrastructure signals well within the per-request perception budget. The pattern borrows MCP's discovery and registration shape. It is not an implementation of the Model Context Protocol wire format, as Section 5 states plainly.
5. **Two-way agent loop**: Escalation outcomes feed back into the edge's own context state within the request life-cycle itself, not only through a slow offline retraining loop that doesn't exist in this design. Together with the dynamic pipeline above, this addresses Failure B. Ablation Run 4 measures the loop's effect directly. Under the retrained policy that effect is small, which Section 8 reports rather than claims as a result.
6. **Policy Orchestration Layer**: This is a governance layer above the fleet. For every client, it decides whether to reuse, fine-tune, or train a new policy. That decision is backed by the task-capacity promotion test (Bossens and Sobey, 2021), and every choice is logged permanently so it can be checked later.

3 Literature Survey

3.1 Confidence- and budget-only cascades

BranchyNet (Teerapittayanon et al., 2016) installs side-branch classifiers on a deep network. A sample can leave early as soon as softmax entropy falls under a threshold chosen ahead of time to hit a target accuracy or speed, not a threshold learned end to end with the rest of the network. This idea has proven very fruitful. Early features are usually very discriminative for easy inputs, and the design is still the basis of most cascading systems built since. But the decision to exit the network is still a function of the input alone. BranchyNet has no cloud queue depth signal, no network round-trip time (RTT) signal, no SLA budget, and no health indicator for the device it runs on. So it simply cannot tell that the next layer is currently overloaded.

MSDNet (Huang et al., 2018) takes the idea further. It supports both "anytime" and "budgeted-batch" classification, spending a fixed computational budget differently on easy and hard parts of an input within one model. This removes the need for separate networks per computation budget, which earlier systems required. But the budget itself is still a value the operator fixes ahead of time, not a measurement of the current infrastructure state. It cannot see the cloud queue growing, or an RTT spike that happened one second ago.

EdgeBERT (Tambe et al., 2021) is the closest developed system to infrastructure-aware routing. It links entropy-based early exit prediction to dynamic power scaling (DVFS) on a per-sentence basis. It holds to a prescribed per-sentence latency target by picking the running frequency that will just clear it. The paper reports targets of 50ms, 75ms and 100ms per-sentence latency. The deadline is an on-device compute-frequency budget. EdgeBERT

never considers offloading to a cloud tier at all, so its state stays limited to the device's own chip. It has no interface to external signals like cloud queue status or network health, because closing that loop was never one of its goals.

SPINN (Laskaridis et al., 2020) splits a single deep network across the edge device and the cloud. It runs the early layers on-device and the rest on a cloud partner. Its scheduler co-optimises the early-exit policy and the split point at run time, reading live network bandwidth and latency alongside device and server load, so as to adapt to changing conditions and meet a user-defined service-level target. This project's own earlier draft description of SPINN as a fixed, split-once system was wrong, and SPINN is in fact the most infrastructure-aware system reviewed in this chapter on a signal-count basis, reading network, device and server state together. What SPINN's evaluation does not show is a persistent, per-device signal carried between requests. Each split-point decision is recomputed from fresh measurements. The paper does not describe a literal per-request SLA-remaining countdown. It only shows a throughput or latency target applied uniformly across the deployment. So SPINN's gap is the absence of edge self-report and outcome memory.

3.2 Learned, single-signal routers

RouteLLM (Ong et al., 2024) trains a router purely from human preference data, to distinguish between a stronger and a weaker LLM. The system shows more than a 2x cost reduction, and it also generalises well to model pairs the router has never seen in training. That is a significant result, and the closest deployed comparison for a learned router. But the only context RouteLLM knows is the prompt itself. It has never seen the concept of serving-system load at all. From a RouteLLM router's viewpoint, a flooded cloud endpoint and a spare one would look the same, and would be routed the same way.

The NSGA-II router (Yu et al., 2025) goes much further toward multi-objective, infrastructure-aware routing. It uses an evolutionary algorithm to balance quality, speed and cost across heterogeneous edge and cloud nodes. It also adapts to the diversity of requests and nodes. The paper's runtime routing algorithm reads a live queue length for each candidate node from a monitoring module, and filters out a node once that queue exceeds a threshold, per request, at dispatch time. NSGA-II computes that threshold, and the rest of the Pareto-optimal policy offline. The paper also describes refreshing it only periodically, hourly or daily in its own example. So the router is genuinely infrastructure-aware at the moment of dispatch, but the policy it applies is a slow-moving snapshot rather than something that learns continuously. It also has no RTT signal and no per-request SLA deadline check, only a queue-depth threshold, and no closed loop feeding individual request outcomes back into the thresholds between re-optimisation cycles.

The PROTEUS model (Bhatti et al., 2026) is the closest in approach to the system built in this project. It is a Lagrangian-constrained RL policy. It accepts a runtime accuracy target as input, routes across a series of LLMs, and enforces a minimum level of accuracy.

The study showed the model reaching accuracy within 1.3% to 4.6% of an oracle, with cost savings of up to 89.8% compared to the best fixed model. That is a very impressive outcome for RL applied to routing. On the downside, PROTEUS's state space contains no information about queues, latency, or energy. The algorithm trades off precision against cost without considering any of the infrastructure signals this project's routing is based on.

GreenServ (Ziller et al., 2026) is the closest of the three learned routers on the energy dimension. It routes each query to whichever model in a heterogeneous pool of large language models has historically balanced accuracy and energy use best for that query profile. It uses a multi-armed bandit that keeps learning online, rather than a policy fixed once at training time. That online adaptivity is exactly what BranchyNet, MSDNet and EdgeBERT lack. The context GreenServ conditions on is mostly on the query side (task type, semantic cluster, text complexity), plus its own model pool's observed accuracy and energy history. It also carries one latency term, but that term is an estimate, not a live measurement. It is derived from the task's configured maximum output-token setting, not from a real-time queue or network reading. So GreenServ still has no channel for cloud queue depth, network RTT, per-request SLA remaining, or the operational health of the requesting device. Two identical requests are routed identically by GreenServ, whether the infrastructure behind them is idle or saturated at that moment, because nothing in its state can tell the difference.

Yang et al. (2025) propose a deep reinforcement learning (DRL) router for LLM serving at the network edge. It is the closest system in this chapter to this project's own approach. Two features set it apart. Its state is built by a heterogeneous graph attention network rather than from a handful of hand-picked features. It also models request interference, where a newly-arrived request's prefill phase slows the decode step of requests already running on that node, which nothing else surveyed here attempts. Its reward is shaped against per-request latency violations, the same objective this project's SLA reward term carries.

What limits it is that every signal it reads sits on the server. The state is queue occupancy and GPU memory utilisation, with nothing about the requesting device's own health or calibration, and the pool it routes across is edge-only. So the edge-cloud escalation decision does not arise for it at all. Its latency reward is a training-time shaping term rather than a deadline evaluated per request. The closest DRL comparator therefore lacks the two things this project adds alongside its own reward shaping: the edge self-report, and SLA remaining read at request time.

3.3 Cluster-aware serving systems

Patel et al. (2024), in their paper on Splitwise, describe the most operationally-aware system reviewed in this chapter so far. It separates a request into phases (compute-bound versus memory-bound) and runs each phase on a machine that specialises in it. Splitwise explains the differences in phase capability, such as throughput and power, so well because it is a real case of heterogeneous server hardware. But scheduling happens at the cluster and phase level. Routing decisions are not checked against a per-request SLA deadline.

3.4 Evidentiary basis of the closest comparisons

RouteLLM's headline "more than 2x cost reduction" needs unpacking before it can be compared like for like. Its core supervision signal is roughly 65K human pairwise comparisons from the Chatbot Arena platform. The 120K/$700 figure often quoted for this router's training data is a separate augmentation set drawn from Nectar and labelled by GPT-4 acting as judge. The cost saving also varies sharply by benchmark: 3.66x on the open-ended MT Bench, 2.49x on MMLU, and only 1.49x on GSM8K at matched response quality. So "more than 2x" describes MT Bench and an aggregate framing. The two closed-label benchmarks, structurally closest to this project's own toxicity-classification task, already sit below the headline figure.

PROTEUS's headline numbers "accuracy within 1.3–4.6% of oracle, cost savings up to 89.8%" are reported for accuracy targets τ restricted to the range $[0.85, 0.95]$, from a single underlying model. The paper demonstrates that a Lagrangian controller can hold a target accuracy floor within that band, but it does not show the same controller holds outside that band, or across a pool of models rather than one. The narrower the τ range and model pool the good numbers were measured on, the more the 89.8% cost-saving figure describes that operating point specifically.

GreenServ's reported 22% accuracy gain and 31% energy reduction are both measured against random routing. Random routing is a weak baseline by construction. Any router that reads even one informative signal about the query beats it by a wide margin. So a gain over random router is a necessary result for a working router. This gain does not prove that its specific bandit formulation does anything more sophisticated than a router with much simpler features would also achieve.

The NSGA-II router's evaluation runs on SQuAD, MBPP, HellaSwag and GSM8K. These cover open-domain QA, code, commonsense reasoning and grade-school maths. None of these is a task where a wrong answer is safety-relevant, or where the "quality" objective the Pareto front optimises over has the unambiguous per-example correctness label a toxicity classification does. The paper demonstrated the ability to balance quality against latency and cost. But it is shown on tasks very different from the closed-label classification problem this project routes. So the comparison in Table 1 should be read as a comparison of routing mechanisms. It should not be read as a claim that the NSGA-II router's own reported numbers would reproduce on this project's task.

None of this weakens the structural gaps identified above. Three of the four (RouteLLM, PROTEUS and GreenServ) still miss a cloud queue depth signal entirely, and none of the four carries an RTT signal or a per-request SLA deadline check, regardless of how their headline numbers were produced. But it is worth saying plainly that "RouteLLM cuts cost by 2x" and "PROTEUS gets within 1.3% of oracle" are results tied to a specific benchmark, preference-data source, or accuracy-target range. They are not general claims this project's own results should expect to match or beat on a like-for-like basis.

3.5 Commercial and open-source landscape

The ten academic systems above are not the only relevant comparison. A mature market of "LLM gateway" products already solves a version of the same routing problem in production. A fair positioning has to account for what they actually do, not just what research papers propose. Two of the most widely deployed products were checked against their own documentation.

LiteLLM (an open-source proxy self-hosted in front of 100+ model providers) documents six routing strategies for its `Router` class. These are simple-shuffle (a weighted pick, the recommended production default), latency-based

(lowest recent response time), usage-based (lowest current token throughput), least-busy (fewest in-flight calls), cost-based, and a pluggable custom-strategy interface. It also has a cooldown mechanism that temporarily stops routing to a deployment after a run of rate-limit or failure responses, and a latency cache that records recent response times per deployment. This makes it a reactive system. It does respond to measured latency and failures.

Even so, every one of its strategies is a fixed rule evaluated against a rolling window of recent responses. None of them is a learned policy. None reads a self-reported edge-device health or calibration state either. Since LiteLLM routes between cloud model endpoints rather than edge devices, there is no analogous self-report to read. And no strategy enforces a per-request deadline before dispatch. A request goes to whichever deployment the current rule favours, and if that deployment is slow, the outcome is only observed after the fact.

Portkey's AI Gateway documents its load-balancing as a static weighted distribution across configured providers or models. Traffic is split according to weights the operator sets, for example for cost optimisation or gradual migration testing. These weights are normalised to sum to 100%. They are not based on a live queue, latency, or health signal measured at request time. Portkey's own documentation for this feature makes no mention of reinforcement learning, closed-loop adaptation from outcomes, or per-request SLA enforcement. Those stay configuration choices the operator makes up front. They are not decisions the gateway itself learns or adapts.

The wider landscape (Envoy AI Gateway, Requesty, and Martian's now-discontinued standalone "intelligence router") occupies similar territory. They offer cost- and latency-aware routing across a pool of model providers, and they are marketed as adaptive. But based on the evidence in each product's own documentation, they are implemented as configured rules or a pre-trained model-performance predictor. They are not systems that learns from live infrastructure telemetry and its own routing outcomes.

OpenRouter is a partial exception, and deserves separating out from that group. Its own documentation for provider routing states that it tracks latency and throughput per provider using percentile statistics calculated over a rolling five-minute window, and deprioritises a provider that has had a significant outage in the last 30 seconds. That is a real live-telemetry signal. It does not rely on a configured rule evaluated once and left alone. Even so, OpenRouter's routing still runs no per-request SLA check before dispatch, and it carries no concept of an edge device at all, since it routes between cloud model providers only. So it narrows the gap on live telemetry specifically, without closing the other two.

None of these products is edge-cloud specific in the way this project is. They route between cloud model providers, not between a resource-constrained edge device and a cloud fallback. So the edge-health self-report, the deadman's switch, and the bidirectional trust loop described in Section 5 have no commercial analogue to compare against at all, OpenRouter included.

This is independently confirmed by Li et al. (2025), a systematic survey of edge-cloud LLM collaboration published within weeks of this project's own evaluation work. It states plainly that it is building "the first systematic foundation" for a unified taxonomy of edge-cloud collaboration. Its taxonomy is organised around task assignment, task division, and mixture-based collaboration, and it contains no category describing a system that combines live infrastructure signals, edge self-report, a closed feedback loop, and a learned routing policy in one deployment.

3.6 Comparison and positioning

Table 1 summarises the twelve systems above, ten from the academic literature and two production LLM gateways. It compares them against three properties that recur through this chapter. Does the routing signal include anything beyond the request itself (infrastructure awareness)? Is a per-request deadline checked before dispatch, not just an aggregate SLA target? Does the outcome of a routing decision change the state the next decision is made from (a closed loop)? No system reviewed combines cloud queue depth, a live per-request SLA-remaining check, an edge device's own self-reported calibration and error state, and a bidirectional outcome loop into one routing decision. Several are strong on a single piece of that list. SPINN re-profiles its split point from live network, device and server load. The NSGA-II router reads live per-request queue depth against offline-tuned thresholds. Splitwise reads cluster-level phase telemetry. GreenServ keeps its bandit learning online. That specific combination of signals, not any one of them alone, is the gap Section 5's design targets.

Table 1: Comparison of cascade, offloading and gateway systems surveyed in this chapter against this project's system.

System	Primary routing signal	Infra-aware	Per-req. SLA	Closed loop	Key limitation
BranchyNet (Teerapittayanon et al., 2016)	Softmax entropy at each exit	No	No	No	Exit decision is a function of the input alone, blind to what is downstream
MSDNet (Huang et al., 2018)	Fixed computational budget	No	No	No	Budget is fixed by the operator ahead of time, not a live measurement
EdgeBERT (Tambe et al., 2021)	Entropy-driven on-chip DVFS	Own chip only	Partial (per-sentence on-device latency target, not a cloud-dispatch gate)	No	No interface to external signals (cloud queue, network), and never considers a cloud tier at all
SPINN (Laskaridis et al., 2020)	Entropy plus live network, device and server load	Yes, live (network, device, server)	No (throughput/ latency target, not a live per-request deadline)	No	Re-profiles from fresh measurements each time, with no persistent edge trust or calibration signal carried between requests
RouteLLM (Ong et al., 2024)	Prompt content, preference-learned	No	No	No	Cannot distinguish a saturated cloud endpoint from a spare one
GreenServ (Ziller et al., 2026)	Query features + model-pool accuracy/energy history + output-token-based latency estimate	No	No	Yes (online bandit)	Latency is an estimate from a configured token cap, not a live reading, and there is still no cloud queue, RTT, SLA or edge-health channel
Yang et al. (2025)	Dynamic per-node queue/GPU state via DRL, with request-interference modelling	Yes, server-side	No (latency-violation term is reward shaping, not a pre-dispatch gate)	No	Edge-only routing, no cloud tier, and the state has no client-device signal
NSGA-II router (Yu et al., 2025)	Live per-request queue depth against offline-tuned Pareto thresholds	Yes, live queue read	No	No	Thresholds refresh only periodically (hourly/daily in the paper), not per outcome, and there is no RTT or SLA-deadline signal
PROTEUS (Bhatti et al., 2026)	Accuracy target τ via Lagrangian RL	No	No (accuracy floor, not a deadline)	No	State space excludes queue, latency and energy entirely
Splitwise (Patel et al., 2024)	Phase-level cluster telemetry	Yes, cluster-level	No	No	Scheduling is cluster/phase granularity, not a per-request deadline
LiteLLM (LiteLLM, 2026)	Weighted/latency/-usage/least-busy rule, operator-selected	Reactive only (latency cache, cooldown)	No	No	Fixed rules over a rolling window, not a learned or self-reported signal
Portkey (Portkey, 2026)	Static operator-set traffic weights	No	No	No	Weights are configured up front, not adapted from live telemetry or outcomes

System	Primary routing signal	Infra-aware	Per-req. SLA	Closed loop	Key limitation
This work	13-slot masked state: confidence, cloud queue, RTT, SLA remaining, edge resource stress, calibration delta, error rate, operational state	Yes, 5 live signals + edge self-report	Partial. Per-request signal, learned soft preference via reward, not a hard gate	Yes, bidirectional loop	Still over-escalates in scenarios where RTT alone already guarantees an SLA violation (Section 7.6)

Three gaps recur across every system in Table 1. Each one maps directly onto a specific component of the design in Section 5. These are not claims added after the literature was read. They were the reason each component was built.

First, most of the confidence- or budget-only cascades (BranchyNet, MSDNet, EdgeBERT) read no live infrastructure state at all. SPINN is the exception among them, reading live network, device and server load to re-profile its split point, but it still stops short of a cloud queue depth or SLA-remaining signal, and it carries nothing self- reported from the edge device between requests. Among the learned and production routers, the NSGA-II router reads a live per-request queue depth, GreenServ keeps an online bandit over its own model pool, and Yang et al.'s DRL router reads a dynamic per-node queue and GPU state. None of these three reads anything about the requesting device itself, only server- and network-side signals. RouteLLM, LiteLLM and Portkey read no live infrastructure signal at all. The Edge Context Protocol and the Bronze/Silver/Gold Medallion pipeline exist to close the remaining gap. They give the routing decision cloud queue depth, RTT, SLA remaining, and the edge device's own resource stress and calibration state, alongside confidence, in one 13-slot vector.

Second, PROTEUS freezes an accuracy-floor policy with no live queue or RTT signal at any point. The NSGA-II routers Pareto thresholds are computed offline and refreshed only periodically. But the per-request routing decision itself does read a live queue-length measurement against those thresholds. Neither system carries a live RTT signal or a per-request SLA-remaining figure. The PPO agent in this project is also trained offline, but its state and reward both carry the current request's own SLA remaining and predicted RTT, at request-time granularity, refreshed every request rather than every hour or day. None of the twelve systems above combines a live per-request queue and RTT reading with an SLA-remaining signal the way this project's Gold vector does. Splitwise comes closest on live telemetry at cluster granularity, not per request, and without an SLA-remaining component. This is still a learned preference the policy weighs against everything else. Section 5 is explicit about that distinction. It even names a Silver-layer signal that was computed for a harder guarantee but never wired into a decision path. So the claim here is request-time granularity.

Third, none of the twelve systems closes the loop back to the requester. EdgeBERT adapts within a single request. SPINN re-profiles its split point from fresh measurements each time. GreenServ adapts its bandit online. LiteLLM adapts its latency cache and cooldown window. But none of them feeds the outcome of one routing decision back into the state the next decision is made from, at the level of a single device. The Bidirectional Loop in Section 5 is built against that gap. Its removal is measured directly, not just asserted, in Ablation Run 4 (Section 7.3).

4 Methodology

4.1 Research approach

This work adopts a Design Science Research (DSR) approach (Hevner et al., 2004). The choice follows from the research question itself. Section 2.1 asks what an active, closed-loop orchestration system buys over a static passive threshold. That question can only be answered by building the artefact and measuring it against the alternatives. It cannot be settled by observation or by survey because no surveyed system combines the properties being asked about (Section 3). This is why DSR approach was choosen.

DSR produces two outputs here. The artefact is the context-aware agentic gateway. The knowledge contribution is the six-item list of features, benefits and capabilities set out in Section 2.3. Each item there is checked against a specific evaluation output, not just declared.

The build-evaluate loop ran three times rather than once. The first trained policy was undertrained and was silently overridden by its own safety fallback. The second was corrected for that but had no reward gradient once an SLA was already violated. The third fitted the **MiscalibrationClassifier** on non-circular data and retrained the policy

under it. Each cycle was triggered by an evaluation result rather than by a change of plan, which is the build-evaluate loop working as intended. All three are reported in Sections 5 and 7 rather than presented as a single clean run.

4.2 Project phases

The project ran over fifteen weeks across ten phases. Table 2 lists the four that map onto the DSR build-evaluate loop. P1, P2, P6 and P10 are supervisor selection, the proposal, infrastructure setup and the poster/viva respectively. These are administrative or infrastructural phases in the loop and they are not repeated here.

Table 2: Phases mapped to the DSR build-evaluate loop.

Phase	Weeks	Name	What it produced
P3	3–5	Literature Review & MDP Formulation	Fed the design: the survey in Section 3 and the MDP formulation in Section 2.2
P4	5–6	Architecture Design & Scope Lock	Fed the design: the Bronze/Silver/Gold schema and the MCP skill interface
P5	6–8	Gymnasium Simulation & PPO Training	Build cycle: the custom Gymnasium environment and the offline PPO agent
P7	9–11	Medallion Pipeline & System Integration	Build cycle: the Bronze, Silver and Gold layers wired into the live request path
P8	11–13	Evaluation & Benchmarking	Evaluate cycle: five core baselines, eight literature baselines, three scenarios, the seven-run ablation (Section 7.1)
P9	9–14	Dissertation Writing	Ran alongside P7 and P8 rather than after them

P5 and P7 are the build cycle. P8 is the evaluate cycle, and its product is the gateway itself.

4.3 Deviations from the plan

DSR expects the artefact to change under evaluation. Six planned choices were not carried through, and each is recorded here rather than left for a reader to notice.

The proposal specified three independent runs per condition. The evaluation reports one run per policy per scenario (**n_runs=1**). Real HuggingFace CPU inference over 1,000 samples for every policy in every scenario made three runs unaffordable inside P8's two weeks, and the choice was to keep real inference rather than keep the repeat count.

The proposal specified energy measurement through Intel RAPL. Energy is instead a simulated proxy computed in **environment.py**. RAPL was never wired in, so no energy number in this report is a hardware power measurement, and Section 7.1 says so at the point the metric is defined.

The proposal specified the PPO agent as a sidecar pod. Both live deployments instead co-locate the agent in the same process as the gateway. Neither is a literal sidecar, and Section 5's Deployment subsection states the distinction plainly.

The proposal specified a TorchServe cloud pod and a Minikube deployment. The cloud pod is a FastAPI service, and the Kubernetes path is a three-region GKE deployment. The single-node EC2 and Minikube path is checked into the repository but has no live-test evidence (Section 6).

The proposal specified F1 as the quality metric. Accuracy is reported instead, because every policy sees the identical capped test set in the identical order, so the class balance is held constant across all comparisons and accuracy separates the policies on the same evidence.

4.4 Data and model selection

The dataset selected for this experiment is Jigsaw toxicity classification (`tasksource/jigsaw_toxicity`, the `comment_text` field against the `toxic` label). It was chosen because it has a real difficulty gradient. A weak and a strong model genuinely disagree on the harder comments, so the routing decision has something to be right or wrong about. A task where both models agree everywhere would make every routing policy look identical.

The edge and cloud pair is **DistilBERT** (`newsmediabias/DistilBert_Toxicity_Classification`) and **BERT-large** (`AnonymousCS/bert-large-uncased-Twitter-toxicity`), both run on CPU. The pair was

chosen so that the accuracy gap is real and the latency cost of escalating is also real. Escalation therefore carries a genuine trade-off rather than a free accuracy gain.

Inference is real HuggingFace CPU inference throughout, so the accuracy figures reflect how these two models actually behave on held-out samples. The dataset is loaded once, shuffled under a fixed seed, and split 70/10/20 into train, validation and test. Each evaluation run draws from the test split at **max_samples=1000**, and every policy sees the same capped set in the same order.

Three metrics, namely Queue depth, RTT and energy are generated by the Gymnasium simulation. Simulating a saturated cloud queue or a sustained 800ms RTT reproducibly on real infrastructure was not affordable, and a simulated signal that can be held identical across every policy is worth more to a controlled comparison than a real signal that varies between runs.

4.5 Validity

Internal validity. Every comparison holds the models, the dataset and the traffic simulation fixed, and varies only the routing policy. A difference in the numbers can therefore only be attributed to the routing decision, a different dataset or different hardware. This is also the reason why the eight literature baselines are re-implementations run through this harness rather than figures quoted from their original papers.

External validity. Training uses domain randomisation. **randomize_scenario()** resamples twelve **ScenarioConfig** fields before every 2,048-step rollout, so the policy sees roughly 49 distinct scenarios across a 100,000-step run rather than one fixed scenario. The held-out scenario (Ablation Run 6) is then set deliberately outside the range those rollouts sample, so a good result there is evidence that the policy learned the decision problem rather than the training scenarios.

Construct validity. Four of the eight recorded metrics are terms in the reward function the policy was trained against. The other four, escalation rate, fallback rate, SLA margin and trust score are not used in the reward function. The second group exists so that a good result can be distinguished from the evaluation simply re-measuring the training objective.

Reliability. Each policy is run once per scenario, so no confidence intervals are reported and no significance test is run. Small differences should be read as indicative rather than established. A gap of one to two accuracy points on 1,000 samples is within the range a single run cannot separate from noise, and this report does not treat such gaps as findings. The claims it does make rest on effects that are an order of magnitude larger, such as escalation falling from 100% to 0.5% and SLA violations from 59.5% to zero under bursty traffic. Those are not differences a repeat run would plausibly reverse.

4.6 Development method and ethics

DSR sets a broad criterion for what counts as valid evidence. It says nothing about how weekly work should be organised inside each phase. That is handled by Scrum (Schwaber and Sutherland, 2020), adapted for a single developer. Sprints are fixed at one week. Each sprint opens with planning that ties a concrete goal to the current project phase and closes with a sprint review. Supervisor meetings run weekly, alternating between mid-sprint check-ins and end-of-sprint reviews. Daily standups are replaced by a personal daily log entry, since there is no team to synchronise. Each week's backlog is allocated to one component or one evaluation run, and the logbook submissions at Weeks 6, 9, 12 and 15 serve as the evidence for each sprint review.

The ethics form was completed and submitted on BREO during P2 (Weeks 2–3). The project involves no human participants and collects no personal data. Jigsaw is a public dataset used under its existing terms, and no new data was collected or redistributed. All models are pre-trained checkpoints obtained from the HuggingFace Hub and used without modification. The synthetic clients in Ablation Run 7 (**client_nhs**, **client_babcock** and the others) are fictional load profiles, not real organisations or their data.

5 Design, Development and Implementation

Figure 1 gives the seven components and how a single request moves through them. The subsections below describe each component in turn.

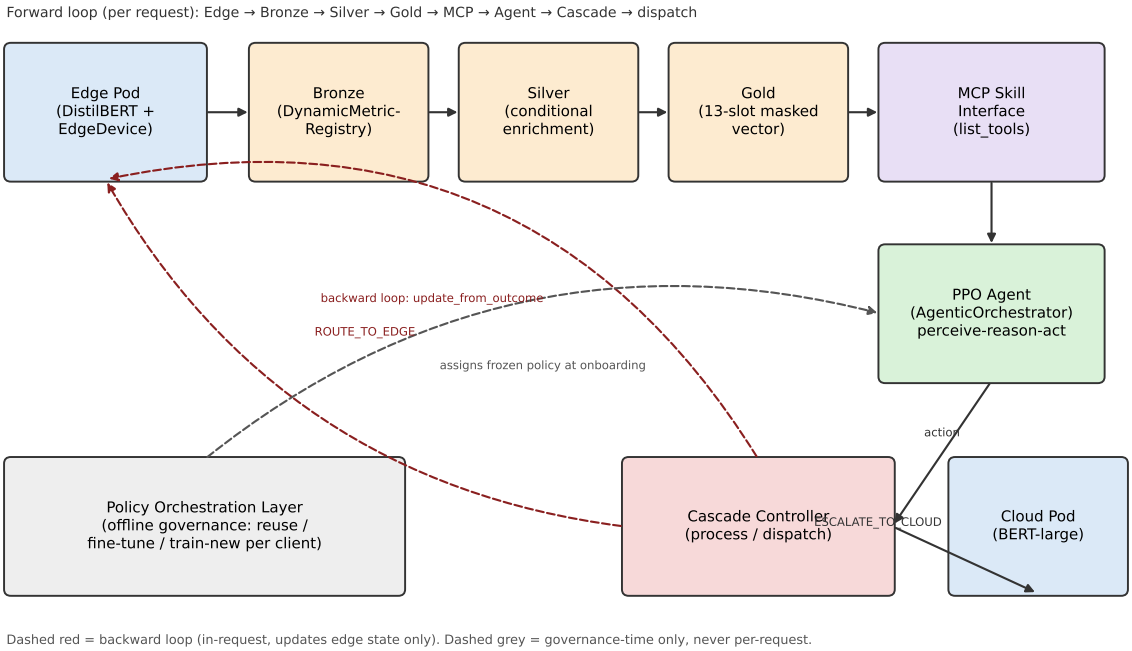


Figure 1: System architecture: the seven components and the request path between them. Solid arrows are the forward (per-request) loop, dashed red the backward loop updating edge state only, and dashed grey the governance-time-only Policy Orchestration Layer.

5.1 Requirements Analysis

The seven components in Figure 1 answers a specific requirement. These requirement comes from the structural gaps named in Section 2.1 (a limited signal set, a static pipeline, a forward-only loop, and the two failures these compound into), and it is confirmed against the literature in Section 3. Table 3 and Table 4 make that derivation explicit. Each requirement traces back to the gap that motivated it. It also traces forward to the component built to satisfy it, and forward again to the evaluation evidence in Sections 6–7 that checks whether it actually did.

Table 3: Functional requirements, traced from the gaps in Section 2.1 to the component that satisfies each one and the evidence that verifies it.

ID	Requirement	Component(s)	Verified by
FR1	Route each request using more than confidence alone: at minimum cloud queue depth, RTT, SLA remaining and edge CPU load	Edge Context Protocol; Dynamic Medallion (Gold vector)	Ablation Run 2 (confidence-only) vs. Run 1, Section 7.3
FR2	The edge device must self-report its own operational health (resource stress, calibration state, error rate) rather than being treated as a passive endpoint	Edge Context Protocol	Ablation Run 3, Section 7.3
FR3	The pipeline must reveal a different feature set depending on the edge's current operational state	Dynamic Medallion (Silver conditional enrichment); MCP Skill Interface	Ablation Run 5, Section 7
FR4	The outcome of a routing decision must feed back into the edge's own state for the next request	Bidirectional Loop	Ablation Run 4, Section 7.3
FR5	A per-request SLA budget must be checked before dispatch	Cascade Controller; Gold **sla_remaining** slot	**sla_violation_rate** / **sla_margin_ms** columns throughout Section 7
FR6	The routing policy must be deployable as a frozen artefact	PPO Agent (frozen **ppo_policy.pt**)	Docker/GKE live multi-region traffic tests, Section 5.9

ID	Requirement	Component(s)	Verified by
FR7	Onboarding a new client or scenario must be a governed reuse/fine-tune/train-new decision	Policy Orchestration Layer	Ablation Run 7, Section 7.4

Table 4: Non-functional requirements, traced the same way as Table 3.

ID	Requirement	Component(s)	Verified by
NFR1	Perception (Bronze → Silver → Gold → **decide()**) must stay a small fraction of end-to-end request latency	All three Medallion layers; MCP Skill Interface	**benchmark_mcp_perception.py**: 0.57ms mean / 0.97ms p95 against 23.9ms mean edge inference (Section 5)
NFR2	The system must fail safe under distributional shift or policy uncertainty rather than trust a possibly-wrong learned action	PPO Agent's OOD/entropy fallback gate (**is_ood()**, entropy > 0.9)	Ablation Run 6 (held-out scenario), Section 7.2
NFR3	Edge unreachability must be detected and handled independently of the learned policy, since a policy that has never seen an absent edge cannot be trusted to reason about it	Deadman's switch (**mark_unreachable()**, **stale_timeout_s**)	Per-request timeout and heartbeat-staleness paths, Section 5.2
NFR4	Governance-time decisions (reuse/fine-tune/train-new, threshold recalibration, LLM diagnosis) must never execute inside the per-request path	Policy Orchestration Layer's explicit offline boundary	Governance-time-only framing and the **_log_decision()** audit trail (**orchestrator_decisions.jsonl**), Section 5.8
NFR5	No feature ships without a traceable link to one of the seven locked components	All seven; this table is the trace	Policy Orchestration Layer's scope-entry record (Week 7–8, traced to supervisor feedback), Section 8

5.2 Edge Context Protocol

5.2.1 Design

ResourceStress records edge health as five separate numbers: cpu, gpu, memory, disk_io, and thermal, each between 0 and 1. CPU and thermal stress degrade confidence calibration, so the model becomes confidently wrong. Memory and disk pressure degrade *latency* instead, which is an SLA risk. A single scalar cannot separate a slow-but-correct edge from a fast-but-wrong one. EdgeBERT (Tambe et al., 2021) is the only system in Section 3 that reads any on-device resource state at all, and even it is restricted to its own chip's entropy/voltage-frequency envelope, with no per-resource breakdown. BranchyNet, MSDNet, RouteLLM, GreenServ, and both production gateways read no device resource state at all. The five-field design is a direct response to that gap.

5.2.2 Implementation

The **EdgeDevice** module acts as an interface that transforms raw readings from device resources into a self-report of its condition. **ResourceMonitor.sample()** takes readings from physical parameters of the device (cpu_percent, virtual_memory().percent, disk_usage().percent) via **psutil**. 'thermal' comes from 'psutil.sensors_temperatures()' and is scaled down assuming a maximum operating temperature of 85°C. If there is no sensor for a certain reading, the system falls back to 0.0. A cloud VM is one example of a device with no sensors. **edgeDevice.apply_stress()** lets the offline simulator inject synthetic values over the same code path that **emit_report()** consumes. This makes the handling of simulated and real stress completely uniform.

EdgeDevice.emit_report() is the core of the protocol. Besides an inference's raw confidence, it computes a calibrated confidence by calling **compute_calibration_delta()**, which applies temperature scaling (Guo et al., 2017). The temperature term is itself a function of the sensed stress ('temp = 1.0 + 0.5*cpu + 0.3*thermal'). So higher stress pulls the calibrated confidence closer to 0.5. This directly targets the failure mode Guo et al. describe, and the one the introduction's problem statement opens with. A model's raw softmax confidence stays high while the prediction underneath it degrades under load. BranchyNet and MSDNet both read raw softmax confidence or entropy as if it were trustworthy at every stress level, with no correction term at all. That is the exact gap

this calibration step closes. **emit_report()** also keeps an operating error buffer (a 60-sample rolling buffer for ‘error_rate‘) and classifies **operational_state** by calling **MiscalibrationClassifier.predict()**.

5.2.3 Classifier status

MiscalibrationClassifier is a logistic regression over the stress vector and error rate. **.fit()** has now been called on real, non-circular data, in two separate domains. Its labels come only from real observed accuracy relative to an idle baseline, never from the fallback heuristic **_fallback_heuristic()** it replaces. That heuristic still stays available as the untrained default. It fires DEGRADING when CPU_thermal > 0.85 or error_rate > 0.10, and STRESSED when CPU_thermal > 0.65 or memory or disk > 0.80.

The first attempt trained on real hardware, under genuine **multiprocessing**-induced CPU stress. It used 80,139 samples and a class-weighted logistic regression, with DEGRADING gated behind a precision/recall-tuned probability threshold rather than plain argmax. This gave 68.0% ± 2.6% balanced accuracy, and 64.4% DEGRADING precision at 66.4% recall, 5-fold cross-validated. But it does not transfer into the Gymnasium simulation. This was confirmed after counting predicted `operational_state` across live simulated steps. It predicts NOMINAL on 100% of inputs because its decision boundary was fit on real **psutil** value ranges that do not overlap the simulation’s synthetic stress distribution.

In the second attempt, it was retrained with the same methodology. The simulation’s own synthetic stress formula was used in the second attempt. It used 27,000samples and the same threshold-tuning approach. This gave 54.6% ± 2.0% balanced accuracy, weaker than the real-hardware fit, particularly on STRESSED precision (33.2%). The same direct check confirms this one does transfer. It produces a genuinely non-degenerate state distribution that rises with scenario stress the way it should. It also shows 2.75% DEGRADING under steady traffic. Under the held-out scenario, it rises to 5.15%. Every evaluation after this section uses this is the classifier by default (**environment.py**, unless **DEVMIND_MISC_CLASSIFIER_PATH** overrides it). Wiring it in also changes routing behaviour by a lot. The policy escalated far more often Under the old heuristic. With the trained classifier, escalation dropped to between 15% and 18% of that old rate in every scenario, while accuracy stayed about the same. Section 7.5 repeats the same swap against the retrained policy, where the gap is much smaller, and separates the two effects. That is because the heuristic’s fixed 0.65 STRESSED threshold sits well inside the simulation’s own “stressed” value range, and fires far more often than the trained classifier does. Full methodology and numbers are in **evaluation/misc-classifier-fit-2026-09-01.md**. It also states a limitation: this behavioural improvement reflects suppressing the heuristic’s false alarms, not a demonstrably more accurate classifier.

5.2.4 Deadman's switch

EdgeDevice.is_unreachable is defined as a property. It stays unreachable until there is a **_last_seen**, and again once ‘time.monotonic() - _last_seen > stale_timeout_s‘. Two independent paths can trigger it. One calls **mark_unreachable()** directly, usually when the edge inference call times out or raises an error. The other is passive. It fires if neither a request nor a **heartbeat()** updates **_last_seen** within the timeout window. **last_report** overwrites the stored **operational_state** to **UNREACHABLE** whenever **is_unreachable** is true. This guarantees no stale values are ever served as current.

The live gateway (‘gateway/app.py‘) executes a **_heartbeat_loop** at 2-second intervals, separate from the request traffic.

None of the eleven systems in Section 3, academic or commercial, has a counterpart to this. Confidence- and budget-only cascades have no concept of edge liveness at all. LiteLLM’s cooldown and Portkey’s failover both react to a request that has already failed, not to a device that has gone silent without failing anything yet. This distinction matters in practice. A deadman’s switch transforms “no data” into a clear state of **UNREACHABLE**. The routing decision can respond to such state even before the next request. It doesn’t have to wait for the request to time out first.

5.3 Dynamic Medallion Pipeline

Figure 2 shows the flow from raw registered sources (Bronze) through conditional enrichment (Silver) to the fixed, masked 13-slot state vector (Gold) that the PPO agent consumes.

5.3.1 Bronze

DynamicMetricRegistry is a dictionary of **MetricSource(name, schema_type, read_fn)** entries. **snapshot()** pulls data from the registered sources and matches names to build the **BronzeMetricSnapshot**. Adding a new

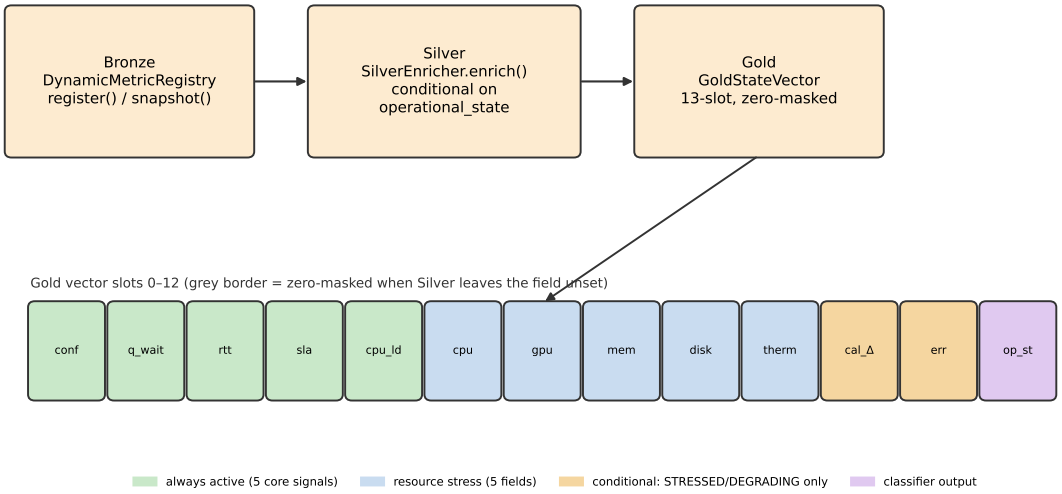


Figure 2: The Bronze/Silver/Gold Medallion pipeline and the 13-slot Gold vector layout, with masking coloured by which slots are always active, which carry resource stress, and which are populated only conditionally.

source only needs **registry.register(...)**. It needs no code changes to '**SilverEnricher**' or **GoldNormalizer**. **cloud_queue_depth** and **rtt_ms** come from the real HTTP client in the live gateway, through **cloud_client.inflight** and **cloud_client.rtt_ms**, as actual measured values from the HTTP client to the cloud pod.

Every system surveyed in Section 3 hardcodes its one signal at design time (BranchyNet's entropy threshold, MSDNet's computational budget, PROTEUS's accuracy target, LiteLLM's and Portkey's operator-set weights). So adding a new kind of signal means changing the routing algorithm itself. A registry entry adds a value without touching **SilverEnricher** or **GoldNormalizer**, which is the direct answer to that limitation. But there is a caveat, extending Gold's 13-slot layout for a new kind of signal still means editing **snapshot()**'s match-case. So the registry is dynamic at the value level.

5.3.2 Silver

SilverEnricher.enrich() computes two predictive features. One is **predicted_queue_wait_ms**, which is an EWMA (alpha=0.3) of the drain rate, **cloud_queue_depth** divided by **sla_remaining_ms**, then multiplied back into **sla_remaining_ms**. The other is **predicted_rtt_ms**, an EWMA (alpha=0.2) of sampled RTT.

This is a computed signal. Tracing every read of **sla_violation_predicted** across the code confirms that it is set here and not read by **GoldNormalizer**, **AgenticOrchestrator.decide()**, or **CascadeController.process()**. **sla_remaining** and **predicted_rtt** are the two Gold slot vectors that reacher the routing decisions. The PPO policy is trained using the reward function's SLA terms. That's why it is a single-request *signal* through which the agent is supposed to learn to act, not a deterministic barrier that blocks an escalation in the way that **sla_violation_predicted** would do. The one hard, rule-based gate in the whole system is narrower, and it lives in the safety fallback. **_resolve_fallback()**'s **queue_backed_up** guard (below) routes to edge when predicted queue wait exceeds a ceiling. But is does not guard in not on every request. It activates only when the fallback path itself is triggered (entropy or OOD). Wiring **sla_violation_predicted** into **decide()** as an unconditional pre-dispatch check would be the harder guarantee the project proposal originally described. That remains open work, not a claimed result.

Enrichment is conditional on **operational_state**, **calibration_delta** is only populated when **STRESSED** or **DEGRADING**, **error_rate** only when **DEGRADING**. The default value is **None**. When the edge is **UNREACHABLE**, **stale=True** is set, which zero-masks the resource-stress slots downstream rather than serving a stale last-known reading.

5.3.3 Gold

GoldStateVector.from_silver() builds a fixed 13-slot layout:

0. **confidence**
1. **predicted_queue_wait**
2. **predicted_rtt**
3. **sla_remaining**
4. **edge_cpu_load**
5. **resource_cpu**
6. **resource_gpu**
7. **resource_memory**
8. **resource_disk_io**
9. **resource_thermal**
10. **calibration_delta**
11. **error_rate**
12. **operational_state**

'mask[10]' and 'mask[11]' are zeroed exactly when Silver left **calibration_delta** or **error_rate** as **None**. **stale=True** additionally zero-masks slots 4 through 10 (edge_cpu_load through resource_thermal). The PPO policy is always fed the full 13-dim vector. Masking communicates *absence*, it does not shrink the input. This is what lets the same trained network handle **NOMINAL**, **STRESSED**, **DEGRADING** and **UNREACHABLE** states without retraining per state.

5.4 MCP Skill Interface

5.4.1 Implementation

The **MCPSkillInterface** in agent.py keeps a dictionary of boolean flags, one per skill, across eight skill groups. Five of these are active by default: **get_model_confidence**, **get_cloud_queue_depth**, **get_edge_rtt**, **get_sla_remaining** and **get_edge_cpu_load**. The other three, **get_edge_calibration_delta**, **get_edge_error_rate** and **get_edge_operational_state**, stay inactive until **register_skill()** is called. **list_tools()** only returns the currently active names. Every skill's **call()** just fetches the relevant element of the Gold vector, which is already computed and stored in memory. There is no network call and no recomputation, so the perception cycle stays cheap by construction.

This was benchmarked directly against real DistilBERT CPU inference (**benchmark_mcp_perception.py**, 300 requests, the same component wiring as the live gateway's **CascadeController**). Perception, meaning Bronze to Silver to Gold to **decide()** including the skill interface, averages 0.57ms (p95 0.97ms). Mean edge inference latency is 23.9ms, so perception is about 2.4% of it, comfortably inside the per-request perception budget.

None of the eleven systems in Section 3 exposes its signals this way. BranchyNet, MSDNet, EdgeBERT and PROTEUS bake their one signal directly into the routing function. Even the two production gateways only expose configuration options to an operator, not a discoverable, callable interface a policy can query for itself at request time.

MCPSkillInterface mimics MCP's discovery and registration shape, a Python class with a dictionary of named, activatable capabilities and a **list_tools()** method. This helps avoid a dedicated http MCP server which will add latency to the request flow.

Earlier designs had **AgenticOrchestrator.decide()** run a second forward pass through the policy after the extended skills were registered, to handle **QUERY_EXTENDED_CONTEXT**. It was replaced with **SilverEnricher.enrich()**. This function computes **calibration_delta** and **error_rate** from the edge's real **operational_state**, already runs before **decide()** does. So the Gold vector the policy sees is identical in both calls, and a second forward pass on the same input adds nothing but random sampling noise from the policy head.

Instead, **decide()** resolves **QUERY_EXTENDED_CONTEXT** by asking **act()** to register the extended skills. It then queries **get_edge_calibration_delta()** and **get_edge_error_rate()** directly, and applies a hardwired risk rule. If either is above threshold (0.2 or 0.15), it escalates to cloud. Otherwise it sends to edge.

One feature remains incomplete: a querying action-dependent masking mechanism, the key of the "query-conditional reveal" ideology. Under it, concealment of the signal would be a function of which skill was queried, not only the operational state. Creating this would mean that the PPO learning loop from which only the raw data is obtained, i.e. **policy.forward()** with which a **decide()** is totally bypassed, would be the only ones to learn the reveal protocol. Such a change was beyond the scope of this project.

5.5 PPO Agent

5.5.1 Training

PPO (Schulman et al., 2017) was chosen over the other standard policy-gradient candidates for reasons specific to this project, not just because PPO is popular.

Take DQN first. The safety fallback in **AgenticOrchestrator.decide()** (Component 4) triggers when action entropy goes above 0.9. That needs the policy to be an explicit stochastic distribution over actions at every decision. DQN's ϵ-greedy action selection over learned Q-values has no equivalent per-decision entropy signal, so the entropy-gated fallback this project depends on would have to be built separately, rather than falling naturally out of the training objective.

SAC is a poor fit for a different reason. Its maximum-entropy machinery (twin Q-networks, a reparameterised continuous policy) is built for continuous action spaces and buys nothing over a 3-action discrete space like this one.

Against vanilla actor-critic (A2C) or TRPO, PPO's clipped surrogate objective approximates TRPO's trust-region constraint without TRPO's second-order Hessian computation, and it is also less sensitive to learning-rate choice than A2C, which matters on a CPU-only training budget with no room for a hyperparameter sweep.

'PPOTrainer''s advantage estimates use Generalized Advantage Estimation (Schulman et al., 2016), the same λ-weighted exponential average of n-step temporal-difference residuals that was introduced alongside PPO's own lineage of trust-region methods. GAE is used here rather than a plain one-step or full Monte-Carlo advantage because the reward is noisy at the level of a single request, a per-request weighted sum of four objectives (accuracy, SLA, energy, latency, see Section 7.1). The λ parameter trades that per-step noise against the bias of bootstrapping from the value head, and that trade-off matters on the 100,000-step training budget this project can actually afford, where a higher-variance estimator would need substantially more rollouts to converge.

'PPOTrainer' ('trainer.py') is a standard clipped-surrogate PPO loop. 'train_agent()' adds domain randomization (Tobin et al., 2017). 'randomize_scenario()' resamples 12 'ScenarioConfig' fields (traffic rates and timing, RTT, edge stress/degrade/unreachable probabilities, cloud service time) from a seeded RNG before every 2048-step rollout, so the policy sees roughly 49 distinct sampled scenarios across a 100,000-step run rather than one fixed scenario.

Domain randomization was chosen over training on a single fixed scenario for a specific reason. The project's target evaluation condition, 'ScenarioConfig.held_out()', is deliberately built to sit outside anything seen in training (Section 7.1). Tobin et al. (2017) show that randomizing simulation parameters widely enough can make a genuinely novel condition look, to the trained policy, like just another sampled variation rather than a distribution shift. Ablation Run 6 tests this claim directly, rather than just assuming it, since it is the run that evaluates the trained policy against 'held_out()' and reports whether performance degrades gracefully or collapses.

An initial 50,000-step run left the policy undertrained. Action entropy on real inference traffic stayed pinned above the 0.9 fallback threshold, so every routing decision was silently overridden by **_resolve_fallback()** instead of the trained policy (**fallback_rate=1.000** across all scenarios). This was found by instrumenting a short training rollout directly. Entropy was declining normally, $1.097 \rightarrow 1.082 \rightarrow 1.028$ over the first three rollouts, an accelerating rate, but it simply had not crossed 0.9 by 50,000 steps. That is evidence of undertraining, not a broken reward or a stuck policy. Doubling the run to 100,000 steps fixed this. **fallback_rate** dropped to 0.000 across all scenarios, confirming the policy's own action is now what actually gets dispatched, not the fixed fallback rules.

A second issue surfaced once the fallback was confirmed to be genuinely off. The retrained policy still escalated to cloud 81.5% of the time under the degraded-network scenario. That made no sense: every policy tested there, including **always_edge**, already hits **sla_violation_rate=1.000**, because RTT alone (800ms) exceeds the 300ms

SLA budget no matter how the request is routed. Escalating in that scenario cannot help. It was paying roughly 4× the latency and 3× the energy of edge-favouring baselines, for no accuracy gain.

The root cause was in the reward function, not the policy. The latency term in **_compute_reward()** (**environment.py**) was **max(0.0, 1.0 - latency/sla_budget)**, floored at zero once latency exceeds the budget. So the reward was identical whether an action's latency was 1ms or 1000ms over budget. That gave the policy no gradient to prefer the less-bad option once the SLA was already lost.

The fix was to remove the floor, using **1.0 - latency/sla_budget** unclamped (the outer **np.clip(reward, -1, 1)** in **step()** still bounds the total). Retraining for a further 100,000 steps reduced degraded-network escalation from 81.5% to 21.5%, energy from 365.0 to 178.4, and p95 latency from 1067.8ms to 950.7ms.

The effect was broader than the narrow case it targeted. Escalation rate fell from roughly 85% to roughly 20% in every scenario, not only the degraded-network one. SLA violations under bursty traffic fell from 65.5% to 0.0%, and energy consumption dropped 50–80% across the board. The accuracy cost was small and consistent, 0.5–2 percentage points per scenario. Full before/after figures per scenario are given in the Evaluation and Discussion section. At this point the degraded-network over-escalation was reduced, but not eliminated. The policy still escalated a fifth of the time, in a scenario where escalating provably buys nothing.

This policy is still not the one the Evaluation and Discussion section reports. The residual over-escalation turned out to be driven less by the reward function than by what the policy was being told about the edge. Once **MiscalibrationClassifier** was fitted (above) and made the simulation default, the policy was retrained from scratch for a further 100,000 steps in that environment. Degraded-network escalation fell again, from 21.5% to 1.5%. That retrained policy is what every number in Section 7 reports. Section 7.5 separates how much of the improvement belongs to the trained classifier and how much to the retraining itself. The residual 1.5% is still reported as a discussed limitation, not a fully closed issue.

5.5.2 Safety fallback

'AgenticOrchestrator.decide()' ('agent.py') checks two independent conditions before it trusts the policy's sampled action. One is action entropy above 0.9. The other is 'is_ood()', a z-score check against saved training statistics ('state_stats.json'). This check requires at least two slots to be simultaneously anomalous ('OOD_MIN_ANOMALOUS_SLOTS=2'), not just one, and it floors each slot's std ('OOD_STD_FLOOR=0.02'). That floor stops a near-constant training signal, for example 'sla_remaining' when the SLA budget never varies in training, from dominating the check on a floating-point-scale deviation.

Either condition routes the decision to '_resolve_fallback()', an ordered list of named guards evaluated first-match-wins. 'queue_backed_up' runs first: it routes to edge if predicted queue wait exceeds a per-instance 'fallback_queue_wait_ceiling', avoiding escalation into an already-collapsing queue. 'low_confidence' runs next, escalating if confidence is below 0.9. Otherwise the default is route-to-edge. The ceiling is an instance attribute, not a class constant, specifically so it can be recalibrated per client without touching the guard list's structure.

5.5.3 Why the fallback is deliberately simple

The fallback is an emergency measure, engaged when the policy's own output is not trustworthy. Relying on it more heavily would mean introducing another learned or more complex system. That would just move the trust problem up a level instead of solving it.

RouteLLM (Section 3) is described purely as a fitted router. A request is routed by whatever the model outputs, with no uncertainty check or out-of-distribution check reported as part of the routing decision itself. The entropy/OOD gate exists so that a bad policy output degrades to a fixed, auditable rule. This is the same pattern that later caught the meta-policy's failure to converge (below), instead of letting it silently make governance decisions.

5.6 Cascade Controller

CascadeController.process() runs the forward pass for a single request. First it applies a 2-second edge inference timeout, calling **mark_unreachable()** on failure. Then it calls **emit_report()** and runs Bronze, Silver and Gold. Then it calls **agent.decide()**, then **_dispatch()**, then **reflect()** and **update_from_outcome()**. Edge inference, edge self-reporting, and the agent's decision all happen in the same process, without crossing a network boundary. The only real network call in the whole request path is when **_dispatch()** calls **CloudClient.predict()** for action **ESCALATE_TO_CLOUD**. **_dispatch()** retries the cloud call once on failure. If it still fails, it uses the already computed edge result (**edge_fallback_cloud_unreachable**) instead of failing the request outright.

5.7 Bidirectional Loop

The forward pipeline: ‘edge_model.predict()‘ → ‘edge.emit_report()‘ → Bronze → Silver → Gold → ‘agent.decide()‘ → ‘_dispatch()‘.

The backward pipeline works like this. **agent.reflect(latency, sla_met, accuracy, tier, fallback)** logs the outcome to the memory buffer. **edge.update_from_outcome(latency, sla_met, accuracy)** updates the edge device’s rolling error buffer and a slow (alpha=0.05) trust EWMA. Only the next request’s Bronze snapshot can see these updates, not the current one. None of these steps change the weights of the PPO policy.

This closes a gap that every system in Section 3 leaves open, not just the cascade baselines. EdgeBERT’s DVFS adapts within a single request, and GreenServ’s bandit adapts across requests, but neither feeds a routing outcome back into the state of the specific device that served it. GreenServ’s update changes which model the *next arbitrary query* gets routed to. It does not change what the edge device itself now reports about its own condition. Ablation Run 4 (Section 7.3) measures the effect of removing this loop directly, rather than leaving its value as an architectural claim.

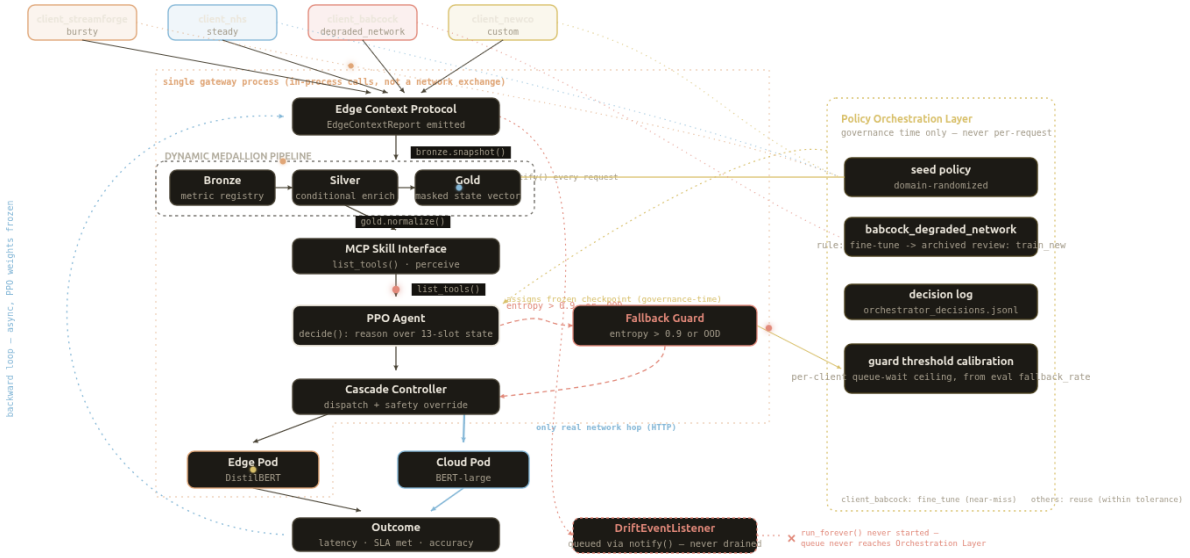


Figure 3: A single frame from the interactive request-flow diagram (**docs/architecture-animated-diagram.html**), showing one request from each of the four client scenarios paused mid-flight at a different pipeline stage: **client_streamforge** approaching the Medallion pipeline, **client_nhs** at the Gold vector, **client_babcock** at the entropy/OOD check into the Fallback Guard, and **client_newco** arriving at the Edge Pod. The right-hand panel is the Policy Orchestration Layer, governance-time only, never per-request. The dashed **DriftEventListener** box marks the same unwired-queue gap Section 5.8.1 discloses in text. The full page animates this continuously and is not reproduced live here.

5.8 Policy Orchestration Layer

The **PolicyOrchestrator** only runs during onboarding and drift detection, not on every request. Its fleet-level value is measured directly here, not just asserted. In a four-client onboarding trial, the governance review escalated one under-provisioned decision that the deterministic rule alone would have accepted (Ablation Run 7, Section 7.4).

The **onboard()** function evaluates every policy in the library against the new client’s scenario, over 3 episodes, using **_evaluate()**. **select_decision()** then chooses between three levels of action. If the client already has a policy that meets all the tolerance thresholds, it reuses it (**REUSE**). If the client’s closest candidate just needs some adjustment, it fine-tunes it (**FINE_TUNE**). If the scenario is completely new, it trains a new policy (**TRAIN_NEW**). That training uses the same domain-randomization function, **randomize_scenario()**, as top-level seed-policy training. So a fine-tuned or newly-created per-client policy is never trained to a lower standard than the shared one.

The REUSE/FINE_TUNE/TRAIN_NEW decision is the task-capacity promotion test described by Bossens and Sobey (2021). It decides whether an existing policy already covers a new task, before spending a full training run on it. Here it is applied at the level of a whole fleet of onboarding clients, rather than a single agent’s sequential task

stream. Neither LiteLLM nor Portkey (Section 3) has anything resembling this. Both route traffic across a fixed, operator-configured set of providers or models, with no mechanism for deciding whether an existing policy already serves a new client well enough, needs adapting, or needs replacing. If that decision exists at all in a production gateway today, a person reading a dashboard makes it, not the system itself.

_log_decision() stores every decision and its supporting metrics to orchestrator_decisions.jsonl. This includes the tolerance gap (**dominant_signal**) that drove the decision, which is one of the most important pieces of the "how and why" explanation this project provides.

5.8.1 Fallback-threshold calibration

calibrate_fallback_ceiling() adjusts the fallback mechanism's client-specific **fallback_queue_wait_ceiling** based on that client's own usage history. It tightens when the client's **fallback_rate** is high (above 0.3), and loosens when the client rarely triggers the fallback (below 0.05). This is the same model 'recalibrate_thresholds()' already uses to govern the reuse/fine-tune/train-new tolerances from historical evidence.

This computed value is not yet consumed live, in the same sense **DriftEventListener** is not (above). **gateway/app.py** builds each client's **AgenticOrchestrator** without reading **orch.fallback_ceilings[client]** back into it. So the calibration is computed and logged, but it does not yet reach the running gateway process. Every such adjustment is logged the same way.

DriftEventListener.should_escalate() looks at two signals together, not just one. A client in sustained distress, reporting **DEGRADING** or **UNREACHABLE** while its trust score stays low, only triggers **onboard()** once that state has persisted for **recovery_window_s**. A spike in error rate overrides this and escalates immediately, as long as the client-specific cooldown has elapsed. Together, these two rules mean a genuine spike is not held back for the full sustained-distress window, and does not re-trigger on every request during an outage.

While extending the governance layer described below, **DriftEventListener** was found to be unwired in the live gateway. **gateway/app.py** never constructs one or passes it to **CascadeController**, which accepts an optional **drift_listener** that stays **None** in production today. Its shape, a non-blocking **put_nowait()** onto an **asyncio.Queue**, consumed by an async **run_forever()**, is correct and is only exercised by its own self-check. It is treated here as built-but-not-yet-live infrastructure, not an active production path.

5.8.2 Per-client thresholds

A company can set its own onboarding bar instead of inheriting the shared default. **PolicyOrchestrator.set_client_thresholds()** stores a per-client **ToleranceThresholds** override. **_thresholds_for(client)** resolves override-or-default everywhere **onboard()** used to read the shared **self.thresholds** directly. **recalibrate_thresholds()** still only drifts the shared default, so a client that set its own bar is not silently moved by the aggregate false-reuse rate observed across other clients. This is exposed on the dashboard through optional **max_sla_violation_rate**, **max_escalation_rate** and **min_accuracy** fields on **POST /clients**.

5.8.3 LLM-backed diagnosis and live monitoring

Two related capabilities were added to make the orchestrator explain itself, not just log a decision. Both call a small local model (Ollama, **gemma4:12b-it-qat**) through **diagnosis.py**'s **DiagnosisProvider** abstraction, entirely off the request path. Neither diagnosis output feeds back into **AgenticOrchestrator.decide()** or any routing decision, so the per-request PPO's determinism claim is untouched.

EscalationDiagnosisMonitor watches per-client action-log rows (the same queue-plus-cooldown shape as **DriftEventListener**). When a client's escalation rate stays ≥95% over a 50-request rolling window, it asks the LLM to explain *why* and *what resource is needed*, and the dashboard broadcasts the result over a websocket alongside a real-time escalation chart, tailed from **request_log.jsonl** by byte offset.

_diagnose_unreachable_threshold() answers a narrower question. After a fresh FINE_TUNE or TRAIN_NEW, if the policy trained specifically for that scenario still misses the client's threshold, that is evidence the requirement itself may be unreachable for that scenario, for example because RTT alone already exceeds the SLA budget. It is not evidence that training just needs another attempt. The same diagnosis provider is then asked whether the requirement is realistic, and what should change.

5.8.4 A learned meta-policy for onboarding decisions, attempted and superseded

The REUSE/FINE_TUNE/TRAIN_NEW decision above is a deterministic threshold rule. To make it learned instead, **orchestrator_trainer.py** built **OrchestrationDecisionEnv**. It runs one synthetic onboarding decision per episode, framed as an episode-length-one MDP, so that **PPOTrainer/PPONetwork** could be reused completely unmodified. The reward is always computed from a genuine evaluation of whichever action was actually taken. REUSE evaluates the real seed policy. FINE_TUNE and TRAIN_NEW actually run a short training run, then evaluate the result. The reward was never a fabricated number, only the training budget during meta-training was reduced, to keep rollout collection tractable. **PolicyOrchestrator._meta_decide()** wired this in with the same entropy/OOD safety fallback that **AgenticOrchestrator.decide()** already uses.

It did not converge. Each step took roughly 47 to 75 seconds, because every step is a real evaluation or a real short training run, not a cheap simulated transition. After 160 real training steps, entropy stayed above the 0.9 fallback threshold on every one of four real test clients. Every governance decision silently fell back to the deterministic rule. A later comparison run produced identical decisions with and without the meta-policy configured, because the learned path never actually engaged.

Matching the per-request PPO's own 100,000-step convergence point, at this measured rate, would take on the order of two months, 54 to 87 days across that range. That is not tractable under this design. The per-request PPO's simulated environment steps are near-free, but this meta-environment's steps are expensive by design, because the reward is never fabricated. Full numbers and logs are kept in **evaluation/option-b-meta-policy-failure-2026-08-24.md**, and the code stays in the repository rather than being deleted. The safety fallback caught the failure exactly as intended, instead of silently authorising a bad governance decision. That is itself evidence the entropy/OOD-gated pattern works, even though this particular learned component's training economics did not.

5.8.5 LLM governance review

This is what replaced the meta-policy attempt. **PolicyOrchestrator._governance_review()** lets the same local LLM review the rule's (or meta-policy's) decision. It can only ever *escalate* that decision to a more cautious action (REUSE → FINE_TUNE → TRAIN_NEW), never downgrade it. So a hallucinated review can waste compute at worst. It can never silently authorise an under-provisioned policy. The review is skipped entirely once the decision is already TRAIN_NEW, since there is nothing more cautious to escalate to, and it is a no-op when no diagnosis provider is configured.

The review was live-tested against a real Ollama instance, using a real client's numbers. A seed policy measured **sla_violation_rate=1.00** against a **0.15** requirement, and the deterministic rule had chosen **fine_tune**. The review correctly escalated the decision to **train_new**, justifying it as "the seed policy's sla_violation_rate of 1.00 significantly exceeds the maximum allowed threshold of 0.15". This took roughly 2.5 seconds once the local model was warm. An earlier attempt against a cold Ollama had exhausted both retries at the raised 20-second timeout, 40 seconds in total. That was an operational quirk of the LLM being idle-unloaded, not a code defect.

A four-client onboarding run with governance review enabled confirmed three clients' REUSE decisions without wasting compute, and escalated the one client whose seed policy missed its SLA requirement by a wide margin. This is discussed further as Ablation Run 7 in the Evaluation and Discussion section.

5.9 Deployment

gateway/app.py and **gateway/cloud_app.py** are two separate FastAPI services. They correspond to an edge-side gateway and a cloud pod. **CloudClient** is the only component that connects these two services, using real HTTP requests. The GCP multi-region deployment script (code/deploy/gcp-up.sh) provisions three VMs: one cloud pod, and two edge-side gateways in different regions. Each edge machine runs its own **CascadeController**, agent, and edge model, all co-located in the same process, so there are no network calls between them. This is similar to a sidecar, but the key point is that the agent here is not a separate remote service adding another network layer. The cloud connection only opens once a request is actually routed to the cloud.

A second, real Kubernetes deployment path exists alongside this one. **code/deploy/gke-up.sh** and **gke-update.sh** provision one small GKE cluster per region (europe-west2 cloud, europe-west4 near-edge, australia-southeast1 far-edge). They deploy the same three services through real **kubectl**-managed **Deployment/Service** manifests (**code/k8s/cloud.yaml**, **orchestrator.yaml**, **gateway.yaml.tmpl**), rather than **docker run** on a bare VM. A single Kubernetes cluster cannot span these three regions, because etcd consensus is not built for the roughly 250ms RTT to the Sydney region. So cross-region calls stay at the HTTP layer, just as in the Docker path above, only

now between GKE clusters instead of bare VMs. Prometheus and Grafana are deployed the same way, as real Kubernetes workloads in the cloud-region cluster (**code/k8s/prometheus.yaml.tmpl**, **grafana.yaml**), scraping **prometheus_client** metrics exposed at **/metrics** on every service across all three regions.

This deployment shape is not something LiteLLM or Portkey are built for. Both route between cloud-hosted model providers, so their deployment unit is just the gateway process itself. Neither has an equivalent to a resource-constrained edge tier running its own model, its own CascadeController, and its own agent co-located with it. Comparing this project against them on "does it beat a production LLM gateway" (Section 3) is fair on the routing-decision axis both address. But the edge/cloud split itself is a problem those gateways were never designed to solve.

5.10 Orchestration pattern classification

Microsoft's Azure Architecture Center publishes a widely-used taxonomy of five multiagent orchestration patterns: sequential, concurrent, group chat, handoff, and magentic (Microsoft, 2026). The same guide is explicit that not every AI system needs to sit at the multiagent level at all. It names a lower complexity tier first: a single agent with tools, which loops through model calls and tool invocations to refine its own result. It recommends using the lowest tier that reliably meets the requirement, because each step up adds coordination overhead, latency, and new failure modes. Placing this project's own design against that taxonomy is useful for a reason beyond comparison. It makes explicit which complexity tier each part of the system deliberately sits at, and why. The classification below is not a single named pattern. It is a hybrid: one pattern for the fast, per-request path, and a different one for the slow, governance-time path, chosen separately because the two paths have different coordination requirements.

The per-request fast path sits at the single-agent-with-tools tier, not the multiagent tier. Bronze, Silver, and Gold (Section 5.3) are deterministic transforms, not agents. They enrich and normalise state without making any decision themselves, so the pipeline that feeds them into the policy is closer to what the same guide calls a sequential pipeline, and closer still to the classic Pipes and Filters pattern it names as sequential orchestration's non-agentic ancestor. One agent sits at the end of that pipeline: the PPO-based **AgenticOrchestrator** (Section 5.5). It reads whichever Gold slots the MCP-pattern skill interface (Section 5.4) currently exposes, exactly the kind of standardised tool discovery the Azure guide itself points to when it names the Model Context Protocol as the mechanism that lets an agent discover and invoke tools. There is no second agent on this path, and no coordination between peer agents to get wrong. That is a deliberate choice, not an oversight. It matches the safety-first, single-pass perception-reason-act design already set out in Section 5.5, and it follows the Azure guide's own advice to avoid the coordination overhead of a multiagent pattern on a path where latency is the whole point.

The governance-time path is where a second, genuinely separate decision-maker appears. The Policy Orchestration Layer (Section 5.8) first proposes a reuse, fine-tune, or train-new decision through its own rule (or, in the superseded experiment, a learned meta-policy). The LLM governance review (Section 5.8.5) then checks that proposal against the same client scenario and can escalate it to a more cautious action. This is a close match for what the Azure guide calls a maker-checker loop, a named sub-pattern of group chat orchestration in which one agent proposes a result and a second agent checks it against defined criteria, pushing back with specific feedback if the result falls short. Two details in this project's version diverge from the pattern as written. First, the check runs once, not as a repeating back-and-forth until the checker is satisfied. Second, the checker can only escalate the proposed decision, never approve something more permissive than the rule itself already chose. That asymmetry is a safety constraint layered on top of the pattern, not a feature the pattern requires, and it exists so a hallucinated review can waste compute at worst rather than silently authorise an under-provisioned policy.

Taken together, this is a hybrid orchestration design, not a single named pattern. Table 5 summarises the split. The fast path stays at the single-agent tier the Azure guide recommends defaulting to, and only the slow, governance-time path steps up to a constrained, one-pass maker-checker loop. The same guide treats a hybrid like this as the expected shape of a real system rather than an awkward compromise: it recommends combining patterns exactly when different stages of one workload have different coordination requirements, rather than forcing every stage into a single pattern chosen up front.

Table 5: The hybrid split: which orchestration pattern each execution path uses, and why.

Execution path	Pattern used	Components involved
Fast path (per-request routing)	Single agent with tools, fed by a sequential pipeline	Medallion pipeline (Bronze/Silver/Gold), MCP skill interface, PPO agent

Execution path	Pattern used	Components involved
Slow path (governance/onboarding)	Maker-checker loop (a group chat variant), one-pass and escalate-only	Policy Orchestration Layer + LLM governance review

6 Testing

Testing here is kept separate from evaluation. This section checks that the components behave as designed. Section 7 is where the assembled system is compared against baselines instead. Two complementary strategies were used throughout development. Both were chosen to fit a project built by one developer on a fifteen-week timeline.

Two further things shaped how this section and Section 7 are structured, not just what they test.

First, this project follows Krishnamachari's (2026) checklist for defensible empirical results. Several items on that checklist map directly onto Section 7's design. Strong, named baselines are used instead of a single straw-man comparison. Five core baselines plus eight literature re-implementations are run against the proposed system (Section 7.1). Multiple regimes are tested instead of one operating point, using three traffic scenarios. And the report states both success and failures of a method. Section 7.6 discloses the degraded-network over-escalation limitation rather than smoothing it over.

Second, self-check correctness and live-traffic testing are kept as genuinely separate activities in this section. A passing self-check suite shows that a component behaves as designed in isolation, but whether the deployed system behaves the same way under real network calls, real models, and real timing is a separate question that self-checks alone cannot answer. Section 6.2 covers what answering that second question required, and what it found that the self-check suite alone had not.

6.1 The self-check convention

Every module in **devmind/** that contains non-trivial logic (a branch, a loop, a numeric threshold, a state machine) carries its own **demo()** function. It can be run directly (**python -m devmind.<module>**), and it asserts the behaviours that would break silently if the logic regressed. For example, **edge.py**'s self-check verifies that **compute_calibration_delta()** returns a signed correction. **agent.py**'s self-check verifies that **is_ood()** needs two anomalous slots, not one, and that a stale **last_fallback_reason** is cleared on the next non-fallback decision. **orchestrator.py**'s self-check verifies, among other things, that a client with an explicit threshold override is not moved by **recalibrate_thresholds()**, and that the LLM governance review can escalate a decision but never downgrade one.

There is no pytest suite with fixtures and mocks. Self-checks are introduced as a small, readable, standalone script. Each of the self-checks is like a small readable standalone script. The test uses the actual methods and real inputs, even very minimal ones at times, and when it fails, it produces a clear and meaningful error that is not just stating the failure itself but also pointing out the reasoning why the invariant matters. Every module touched during this project's latervsprints carries one of these self-checks (**edge.py**, **dataset.py**, **cascade.py**, **agent.py**, **orchestrator.py**, **diagnosis.py**, **orchestrator_trainer.py**, **gateway/orchestrator_app.py**), and none of them makes a real network call. External dependencies, HuggingFace Hub and Ollama, are stood in for with small fake objects that match the real interface. For example, **orchestrator_trainer.py**'s **_FakeModel** matches **DistilBERTEdge/BERTLargeCloud**'s **.predict(text, true_label)** signature without calling a real model. So the whole suite runs in well under a minute, and it can be re-run after any change without needing a live model or a network connection.

Two bugs were found and fixed directly through this discipline. **compute_calibration_delta()** in **edge.py** was subtracting an unsigned gap from raw confidence, instead of applying the signed, temperature-scaled correction. And **TextClassificationDataset.shuffle_train()** in **dataset.py** was shuffling a throwaway filtered copy of the training splits instead of the samples actually used for training. Both bugs were caught while writing the self-check for each module, during a full read-through of the codebase. Neither was caught by a failing test that had already shipped.

6.2 Live integration tests

Self-checks deliberately avoid real models, HTTP connections, and LLM calls. So a second layer of testing exercised the system with all three present, for the parts where a fake cannot stand in for the real failure surface.

- **Multi-region live traffic test** (documented in **evaluation/multi-region-live-traffic-test-2026-08-05.md**). The three Docker services (edge gateway, cloud pod, orchestrator dashboard) were deployed to real GCE VMs across three regions via **deploy/gcp-up.sh**, and driven with real traffic. This is what surfaced a genuine gap that no unit-level test had caught. The cloud dispatch path in **cascade.py** had no **try/except** around the network call, unlike the edge path, which does. This has since been fixed. **_dispatch()** now retries the cloud call once inside a **try/except**, then falls back to the already-computed edge result if it still fails (Section 5, Cascade Controller). That is exactly the class of gap a live multi-region test can surface, and a self-check running against local fakes cannot.
- **LLM diagnosis capability**. This was tested end-to-end against a real local Ollama instance (**gemma4:12b-it-qat**), not just the scripted stub used in self-checks. A synthetic **client_babcock** distress scenario produced a coherent, correctly-parsed diagnosis in roughly 2.5 seconds once the model was warm. A first, cold-Ollama attempt timed out at the then-configured 15 seconds. That is why the provider's default timeout was raised to 20 seconds.
- **Websocket dashboard**. This was tested with a real **uvicorn** server, a real websocket client, and temporary action-log files, not just the in-memory fakes (**FakeWS**, **DeadWS**, **FakeMonitor**) used in **orchestrator_app.py**'s self-check.
- **LLM governance review**. This was live-tested against real Ollama, using a real client's numbers (Section 5's "LLM governance review"). It correctly escalated a **fine_tune** decision to **train_new**, given a genuine large SLA-violation gap, and gave a coherent justification for doing so.
- **Multi-region Kubernetes deployment** (documented in **evaluation/multi-region-gke-live-traffic-test-2026-09-02.md**). The three services were deployed via real **kubectl**-managed **Deployment/Service** manifests, onto one small GKE cluster per region. A single cluster cannot span the ≈250ms RTT to the Sydney region, so cross-region calls stay at the HTTP layer, the same as in the Docker path. This deployment ran the classifier-retrained policy (Section 7.5), and handled 9,835 requests with zero errors and zero pod restarts across the full session.

  The live escalation rate tracks the matching offline condition closely. For **client_babcock** it was 2.5% live against 3.5% offline, and the other clients were within 0.2–0.7 points. These differences are explained by real network variance, not a regression. The matching offline condition uses the retrained policy paired with the untrained heuristic, not the trained classifier. **gateway/app.py** only reads a fitted classifier when **DEVMIND_MISC_CLASSIFIER_PATH** is set, and this deployment deliberately left it unset. That classifier was fitted on the simulation's synthetic stress distribution, and it has never been validated against real **psutil** readings (Section 7.5). So the live gateway and the offline simulation default to different classifiers. This is a disclosed scoping decision, not an oversight, and Section 7.5's comparison is what makes it a defensible one. The retrained policy turns out to be nearly insensitive to which of the two feeds it.

  This deployment also ran Prometheus and Grafana as real Kubernetes workloads, scraping **prometheus_client** metrics across all three regions. It surfaced two further gaps that a self-check running against local fakes cannot catch. First, **prometheus_client**'s default **Histogram** buckets are calibrated for second-scale durations, and they silently degraded millisecond-scale latency quantiles until explicit buckets were set. Second, the orchestrator's live-monitoring view tails a local log file, and it needed an explicit HTTP-forwarding path (**CascadeController**'s **action_log_url**) once the gateway and orchestrator became separate pods rather than processes sharing one host. Both problems were fixed and verified against live cross-region traffic.

  One constraint remains, and it is disclosed rather than hidden. The GCP project's global **IN_USE_ADDRESSES** quota is 8, and this deployment's own 8 services already hold all of it. So Jaeger cannot also acquire an external IP. The pod runs, but its UI and OTLP endpoint are not externally reachable under this project's current quota.

6.3 What was not tested

The single-node EC2+Minikube deployment path (**deploy/ec2-minikube-setup.sh**) is checked into the repository. It looks plausible on inspection, but it has no live-test report to back it up. The 3-region GKE deployment above is the Kubernetes path that does have live-test evidence, using genuine multi-cluster Kubernetes rather than a single local cluster.

The learned orchestrator meta-policy (Section 5) was tested in one limited sense. Its integration code, safety fallback, and training loop all pass self-checks, and were run against real training data. But it was never tested in the sense of

producing a trusted live decision, because it never converged. That is reported here as a failure, not as a gap in test coverage.

7 Evaluation and Discussion

All results below are the final run (**evaluation/2026-09-02_00-47-07.json**), produced by the policy retrained from scratch for 100,000 steps inside the classifier-default environment described in Section 7.5, with one run per policy, **max_samples=1000**, real HuggingFace CPU inference throughout. Runs 5 and 7 were regenerated against that same policy and are recorded separately, in **evaluation/run5-silver-ablation-retrained-2026-09-02_01-22-39.json** and **evaluation/run7-governance-retrained-2026-09-02_01-26-14.json**, because they are driven by their own harness entry points rather than by the main sweep.

Three earlier runs are kept in the same directory for traceability. They are cited only where the contrast itself is the finding, never as headline numbers here. **evaluation/2026-08-20_06-40-51.json** records the undertrained policy, at **fallback_rate=1.000** in all three scenarios. **evaluation/2026-08-20_08-42-51.json** records the state just after that undertraining was fixed but before the reward-gradient fix. Its degraded-network row supplies the 81.5% escalation, 365.0 energy and 1067.8ms p95 figures quoted in Section 5. **evaluation/2026-08-23_21-08-21.json** was the headline run before the classifier retrain, and it is fully superseded by the run named above. Section 7.5's "heuristic" rows come from their own comparison log instead, and are cited there.

7.1 Evaluation strategy

The evaluation is built around one design principle. Every comparison below holds the models, the dataset, and the traffic simulation fixed, and varies only the routing policy. So a difference in the numbers can only come from the routing decision itself, not from a different model, a different dataset, or different hardware. This is why the reported numbers are not the original papers' own published figures. Section 3 already establishes those are not directly comparable, since they use different datasets, different tasks, and different hardware. Instead, the numbers come from running each policy, including faithful re-implementations of eight literature policies, through the identical harness described below. The two production LLM gateways are compared qualitatively only. They are not run through this harness, and the disclosure later in this subsection explains why.

7.1.1 Metrics

Every run produces one **EvalMetrics** record (**evaluation.py**). **run_episode()** computes it from a single deterministic pass over the task's test split, one request at a time. That split is capped at **max_samples** samples when the environment is constructed (**load_task_dataset()**, **environment.py**). The episode ends once **total_requests** ≥ **len(test split)**, so every policy sees the same capped set of requests in the same order:

Table 6: EvalMetrics fields recorded for every evaluation run, and the rationale for each.

Metric	Definition and rationale
accuracy	Mean correctness (edge or cloud model's prediction against the ground-truth label) over every served request. The primary quality signal, and every routing decision that avoids the more accurate cloud model risks this number.
p95_latency	95th percentile of per-request latency (edge inference time, or cloud queue-wait + RTT + cloud inference time, depending on the action taken). The 95th percentile is used instead of the mean because SLA compliance is a tail-latency problem, not an average-case one. A policy with a low mean but a heavy tail still fails users at the tail.
sla_violation_rate	Fraction of requests whose latency exceeded **scenario.sla_budget_ms** (300ms by default). This is the metric a per-request SLA check is specifically designed to keep low. It is the direct evidence for or against the SLA-budget contribution claimed in Section 2.3.

Metric	Definition and rationale
escalation_rate	Fraction of requests routed to the cloud. Read alongside SLA violation and accuracy, not alone: a policy can trivially drive this to 0 (always_edge) or 1 (always_cloud). It only becomes informative in combination with the other metrics.
energy_mj	Cumulative energy proxy summed over the episode: an edge request costs 0.5 + 0.3 × cpu_stress, a cloud request costs 2.0 + 0.1 × cloud_queue_depth (both in the same proxy units, **environment.py**'s **step()**). This is a simulated proxy, not a measurement from real hardware power counters (Intel RAPL was named in the original proposal but never wired in, see Limitations). It captures the right qualitative pattern (queue-length-dependent cloud cost, stress-dependent edge cost) without claiming to be a calibrated joule measurement.
fallback_rate	Fraction of requests where the entropy/OOD safety fallback (Section 5) overrode the policy's own sampled action. This is the metric that caught the undertrained-policy bug (Section 5): a **fallback_rate** of 1.000 means the reported numbers are actually the fixed fallback rule's behaviour, not the trained policy's, regardless of which policy name is on the row.
sla_margin_ms	Mean of the edge device's own self-reported **sla_margin_ms** (Section 5) across requests where an edge report exists. Distinct from **sla_violation_rate**: this is the edge's own local claim about SLA feasibility, not the ground-truth outcome, so a persistent gap between the two is itself diagnostic of miscalibration.
trust_score	Mean of the edge device's slow EWMA trust signal (Section 5) across the episode. Read as a sanity check on the Bidirectional Loop specifically: if it never moves across an episode, the backward loop is not doing anything observable in that scenario.

These eight metrics are not independent of the training objective. Four of them (accuracy, latency, SLA compliance, energy) are the four terms the reward function in **environment.py** optimises during training:

$$r = w_{\text{acc}} \cdot \text{accuracy} + w_{\text{sla}} \cdot (1 \text{ if sla\_met else} -0.5) + w_{\text{energy}} \cdot \left(1 - \frac{\text{energy}}{5.0}\right) + w_{\text{lat}} \cdot \left(1 - \frac{\text{latency}}{\text{sla\_budget}}\right) - \text{perception\_cost}$$

clipped to $[-1, 1]$. The weights $(w_{\text{acc}}, w_{\text{sla}}, w_{\text{energy}}, w_{\text{lat}})$ are normalised from **ScenarioConfig.reward_weights**, with default $(0.4, 0.3, 0.2, 0.1)$. **perception_cost** is a small fixed penalty (0.02, **environment.py**). It is charged only when the agent chose **QUERY_EXTENDED_CONTEXT**, and it is subtracted after the four weighted terms, before the outer clip.

Evaluating on the same four axes the policy was trained against is deliberate. It lets a result be read two ways. Either the policy generalised its training objective to unseen traffic, or it overfit to the reward, and the evaluation is just re-measuring that same reward. The remaining four metrics (escalation rate, fallback rate, SLA margin and trust score) exist to tell those two readings apart. None of them is a reward term the policy was ever optimised against directly.

7.1.2 Scenario design

Four **ScenarioConfig** presets (**environment.py**) each stress a different axis of the state space. That way, a policy's failure mode can be traced to a specific kind of infrastructure stress, rather than just "traffic in general":

Table 7: The four `ScenarioConfig` presets used in evaluation, and the infrastructure axis each isolates.

Scenario	Request rate	RTT	Edge stress prob.	What it isolates
Steady	4,000/min flat	40ms flat	0.10	Baseline: no stress axis active, tests whether the policy adds unnecessary overhead when nothing is wrong
Bursty	4,000→22,000/min, burst covers 20–70% of the episode	40ms flat	0.10	Cloud queue depth alone. RTT and edge health are held at their steady values
Degraded network	4,000/min flat	40→800ms for the full episode	0.10	RTT alone. Deliberately holds request rate and edge health at steady values so a poor result cannot be blamed on volume
Held-out (Ablation Run 6)	2,000→30,000/min, burst covers 15–75%	100→1,200ms for the full episode	0.40 (degrade prob. 0.15)	All axes simultaneously, at a severity never sampled during domain-randomised training. Tests generalisation, not memorisation of the training distribution

The held-out scenario is not a fifth entry in the same family. Every field in it is deliberately chosen to sit outside the range that **randomize_scenario()** (**trainer.py**) samples during training. So a good result there is evidence the policy learned the underlying decision problem, not just the training scenarios themselves.

7.1.3 Baseline design

Baselines fall into three tiers, structured so each tier answers a different question:

Five core baselines (**always_edge**, **always_cloud**, **static_tau_0.6**, **static_tau_0.95**, **random**) establish the floor and ceiling of the decision space. The best any policy can do without reading anything beyond confidence is a threshold on **static_tau**, so the proposed system has to beat these to justify the extra machinery at all.

Eight literature baselines are not the original authors' code, trained weights, or datasets. None of those eight papers evaluated on the Jigsaw toxicity task with a DistilBERT/BERT-large edge/cloud pair, so citing their own published numbers here would compare different tasks and hardware, not different routing policies. Each is instead a faithful re-implementation of that paper's *routing rule*. It is run through the exact same harness (same models, same dataset, same traffic simulation) as every other row in the results tables that follow. That way, the only variable being compared is the policy itself:

Table 8: The eight literature baselines re-implemented as routing rules and run through the same harness as every other policy.

Baseline	Original mechanism (Section 3)	This project's re-implementation (**evaluation.py**)
branchynet_entropy	Exit early once softmax entropy falls under a threshold	Computes binary entropy from the Gold vector's confidence slot, escalates if entropy $\geq \tau$
msdnet_budget	Spend a fixed computational budget, more on hard inputs	Escalates if confidence $< \tau$ *and* the predicted RTT is still within a fixed latency budget
edgebert_dvfs	Entropy-driven exit linked to on-chip DVFS	Escalates if confidence $< \tau$ *and* edge CPU load is below a fixed limit (a CPU-load gate standing in for the voltage/frequency envelope, since this project has no physical DVFS-capable accelerator)

Baseline	Original mechanism (Section 3)	This project's re-implementation (**evaluation.py**)
spinn_-connectivity	Route based on entropy plus network connectivity	Escalates if confidence $< \tau$ *and* predicted RTT is within a fixed connectivity limit
proteus_-accuracy_floor	Enforce a minimum accuracy target via Lagrangian RL	Escalates whenever confidence is below a fixed accuracy target (the accuracy-floor *behaviour*, not the Lagrangian training procedure that produces it in the original paper)
splitwise_-queue_energy	Phase-aware scheduling under queue and power constraints	Escalates only if predicted queue wait *and* average resource stress across all five stress channels are both below fixed limits
routellm_-learned_router	A router trained from preference data to pick between a strong and a weak model	A real **sklearn.LogisticRegression**, fitted on this project's own simulated rollout data (confidence → correctness), mirroring RouteLLM's learned rather than rule-based approach
nsga2_pareto_-rule	Evolutionary multi-objective optimisation over quality, latency, and cost	A grid search over 18 threshold values that selects whichever τ maximises a weighted accuracy/SLA/energy score. The multi-objective *outcome* NSGA-II is built to reach, found here by search rather than a genetic algorithm

The two production LLM gateways (Section 3) are not in this table, and this is disclosed rather than implied. LiteLLM's and Portkey's routing strategies were checked against their own documentation, and are compared *qualitatively* in Section 3's comparison table. But neither was re-implemented as a callable policy in **evaluation.py** and run through this harness. Tracing every baseline registered in **evaluate_literature_baselines()** confirms this directly.

Doing this fairly would need more care than the eight academic baselines above did. LiteLLM and Portkey route between cloud model *providers*, not between an edge device and a cloud fallback. So their rules would first need a deliberate translation decision, for example treating **always_edge** as one "provider" and **always_cloud** as another for a weighted split. That translation has genuine room to be done unfairly to either side. It was not attempted under this project's timeline, so no empirical row is claimed for either gateway here. The comparison against them rests on the qualitative evidence in Section 3, not a number in this section. Adding a fair empirical version is named as a concrete next step in the Conclusion.

7.1.4 Ablation design

Each ablation run changes one component relative to **run1_full**. That way, a difference in its numbers isolates that one component's contribution, rather than mixing several changes together:

Table 9: The seven-run ablation design: each run changes one component relative to `run1_full`.

Run	What differs from run1_full	What it isolates
1 (**run1_full**)	Nothing. The proposed system: all 5 standard signals, full bidirectional loop	The reference point every other run is measured against
2 (**confidence_-only**)	A fixed $\tau = 0.6$ threshold on confidence alone, no other signal	The RouteLLM-equivalent condition: what a single-signal router gets under the same traffic
3 (**calibration_-delta**)	Confidence threshold plus a fixed override (**calibration_delta** > 0.15 forces escalation)	Whether the Edge Context Protocol's calibration signal helps *as an add-on to a threshold rule*, isolated from whether it helps when fed to a learned policy (run1_full already answers the latter)
4 (**no_-reflect**)	**run1_full**'s trained policy, but **env.use_reflect=False** disables the backward loop	The Bidirectional Loop's contribution, with everything else (policy, signals, scenario) held identical

Run	What differs from run1_full	What it isolates
5 (**silver_modes**)	**run1_full**'s trained policy, unchanged, run against three Silver enrichment strategies: static (always reveal **calibration_delta/error_rate**), conditional (the shipped, **operational_state**-gated behaviour, equivalent to run1_full), and agent-driven (revealed only on a **QUERY_EXTENDED_CONTEXT** action)	The dynamic Medallion pipeline's own contribution, isolated from the policy or the scenario
6 (held-out)	Same trained policy, run on **ScenarioConfig.held_out()** instead of the three main scenarios	Generalisation beyond the domain-randomised training distribution
7 (governance layer)	The **PolicyOrchestrator**'s per-client reuse/fine-tune/train-new decisions, compared against one shared frozen policy served to all four clients	The Policy Orchestration Layer's value at fleet level, described in Section 5 and discussed separately below since it is evaluated through **orchestrator.py**, not **evaluation.py**'s single-policy harness

Run 5 (Silver modes). The evaluation plan also specified a three-way comparison of static, conditional, and agent-driven Silver enrichment. This isolates the dynamic Medallion pipeline's own contribution, the same way Run 4 isolates the bidirectional loop's. All three conditions reuse **run1_full**'s frozen policy, unchanged. Only what **SilverEnricher.enrich()** reveals to it varies. **Static** always sets **calibration_delta** and **error_rate**, regardless of **operational_state**. **Conditional** is the shipped behaviour. It applies the same reveal rule as **run1_full**. Its numbers still differ a little from Table 10's, because Run 5 is a separate pass through the harness, not because the two behave differently. **Agent-driven** withholds both signals by default. It reveals them only when the policy's own first-pass action is **QUERY_EXTENDED_CONTEXT**. At that point, **InferenceGatewayEnv.peek_extended_silver()** recomputes Silver with full reveal over the same Bronze snapshot. This is a genuine second look, one that **AgenticOrchestrator.decide()** itself cannot take. It runs one forward pass per request, and resolves **QUERY_EXTENDED_CONTEXT** with a fixed rule instead (Section 5). This eval-only scaffolding gives the agent-driven condition the two-pass behaviour that decide() is documented as not having, without changing decide() itself.

The differences across the three conditions are real, but modest. The direction is not even consistent across scenarios. Static Silver has the lowest energy under steady traffic, but the highest under bursty and degraded. Agent-driven has the lowest p95 latency under steady (120.7ms), but the highest under bursty (184.2ms). No condition dominates the other two across all three scenarios. **fallback_rate=0.000** throughout confirms the policy handled all three reveal regimes without tripping the entropy/OOD safety net, including the two it was never trained under. This is evidence that the 13-slot masked representation generalises across different masking *patterns*, not just masking *amounts*. But the effect size is small enough that this should be read as a robustness finding, not as a case for preferring one Silver strategy over another based on these three scenarios alone.

7.1.5 Reproducibility

Every table in this section comes from **save_results()**. It writes both a human-readable **.txt** summary and a machine-readable **.json** record (per-policy **accuracy**, **p95_latency_ms**, **sla_violation_rate**, and the remaining **EvalMetrics** fields via **_metrics_to_dict()**) to **evaluation/**, timestamped when the run completed. The headline numbers below come from one specific, named file (**2026-09-02_00-47-07.json**, referenced at the top of this section), not a cherry-picked run among several. Runs 5 and 7 are taken from the two companion files named there.

Every other timestamped file in the same directory is either superseded by those, or cited explicitly by name where its contrast is itself the finding. That covers the undertraining and reward-gradient bugs (Section 5), and the untrained-heuristic comparison (Section 7.5). The fitted-classifier artefacts in the same directory (**.joblib** models, their **-meta.json** metrics, and the cached **.npz** training data) are inputs to Section 7.5, not results in their own right. Each policy is run once per scenario (**n_runs=1**, **max_samples=1000**) against real HuggingFace CPU inference, not a simulated accuracy model. So the accuracy numbers reflect the genuine DistilBERT/BERT-large edge/cloud pair's real behaviour on held-out Jigsaw samples, not a synthetic approximation of it.

7.2 Baselines and scenarios

The proposed system is **run1_full** which is the PPO agent described in Section 5. It consists of all five standard signals and the full bidirectional loop. It is compared against five core baselines (**always_edge**, **always_cloud**, **static_tau_0.6**, **static_tau_0.95**, **random**) and eight literature baselines implemented in **evaluation.py** (**branchynet_entropy**, **msdnet_budget**, **edgebert_dvfs**, **spinn_connectivity**, **proteus_accuracy_floor**, **splitwise_queue_energy**, a fitted logistic-regression **routellm_learned_router**, and **nsga2_pareto_rule**). Three traffic scenarios are used here. They are: 1. a baseline steady scenerio with 4,000 requests/minute. 2. a burst scenerio ranging from 4,000 up to roughly 22,000 requests/minute 3. a degraded-network scenario that holds RTT at 800ms for the full run.

7.2.1 Steady traffic

Table 10: Results under steady traffic (4,000 requests/minute flat).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.795	73.5	0.000	0.000	119.1
always_cloud	0.805	171.6	0.000	1.000	420.0
static_tau_0.6	0.805	179.5	0.000	1.000	420.0
static_tau_0.95	0.805	175.8	0.000	1.000	420.0
random	0.780	167.5	0.000	0.475	259.8
routellm_learned_router	0.790	75.6	0.000	0.000	117.5
run1_full (proposed)	**0.785**	**75.8**	**0.000**	**0.010**	**120.0**

Under steady traffic, every policy holds zero SLA violations. So the columns that actually separates policies here are escalation rate and energy,. **run1_full** escalates just 1% of the time which keeps it close to **always_edge**'s cost profile (75.8ms p95, 120.0mJ against 73.5ms/119.1mJ). It achieves this feat while still keeping the option to escalate on the rare request that actually needs it. **static_tau_0.6** reaches marginally higher accuracy (0.805 against 0.785), but only by escalating on every request and spending 3.5× the energy. When there is no infrastructure stress, a continuous 13-dimensional policy surface does not have much to differentiate on. The policy correctly learns to behave close to the cheap edge-only baseline, instead of escalating needlessly.

7.2.2 Bursty traffic

Selected rows. The complete per-policy results for all three scenarios are in **evaluation/2026-09-02_00-47-07.txt**.

Table 11: Results under bursty traffic (4,000→22,000 requests/minute).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.790	71.9	0.000	0.000	131.0
always_cloud	0.805	628.4	0.595	1.000	994.8
static_tau_0.6	0.805	626.5	0.605	1.000	994.8
routellm_learned_router	0.790	78.7	0.000	0.000	139.3
run1_full (proposed)	**0.790**	**76.8**	**0.000**	**0.005**	**136.5**

This is the headline result. Under burst conditions, every static or always-cloud baseline that tries to preserve accuracy pays for it with SLA violations in the 59.5–60.5% range. **run1_full** holds **zero** SLA violations. It escalates on only 0.5% of requests, at 14% of **always_cloud**'s energy cost, while matching its accuracy to within one and a half percentage points. The two edge-favouring baselines (**always_edge** and **routellm_learned_router**, which converged to a near-always-edge policy) avoid SLA violations too, and match **run1_full**'s own accuracy exactly, all three at 0.790. But **run1_full** still keeps the option to escalate on whichever requests genuinely need it, instead of committing to one fixed strategy. This is the main design challenge for the entire architecture. The setting takes an adaptable routing function in the presence of infrastructure stress, and the baselines it beats are naive rules and a learned router fitting.

Figure 4: SLA violation rate and escalation rate per policy under bursty traffic (Table 11). Every static or always-cloud baseline that preserves accuracy pays for it with SLA violations in the 59.5–60.5% range. while `run1_full` holds zero at half a percent of the escalation rate.

7.2.3 Degraded network

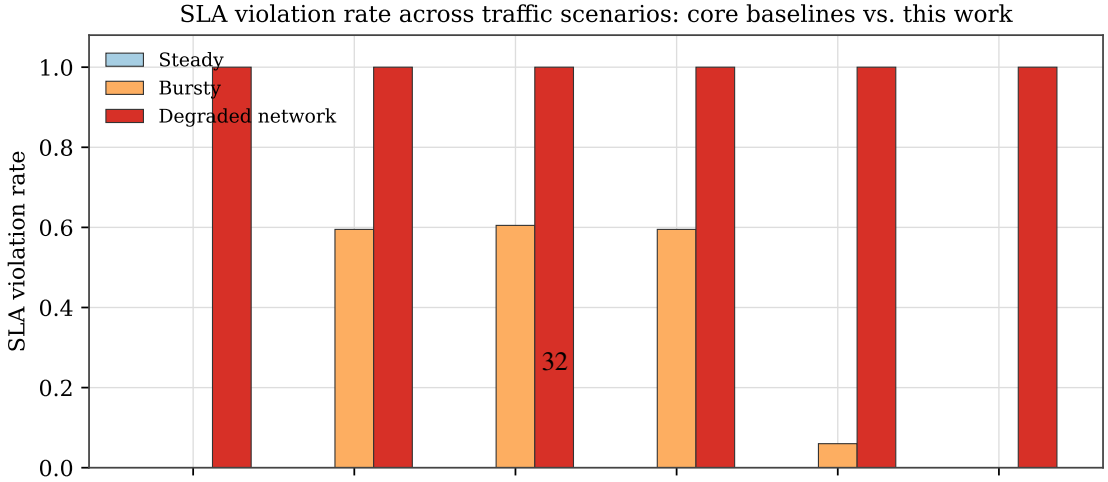
These rows are a selection. The full per-policy results for all three scenarios are in **evaluation/2026-09-02_00-47-07.txt**.

Table 12: Routing results under degraded-network conditions where RTT held at 800ms for the full episode.

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.790	454.6	1.000	0.000	114.3
always_cloud	0.805	941.7	1.000	1.000	420.0
msdnet_budget	0.790	453.1	1.000	0.000	115.1
spinn_connectivity	0.790	451.7	1.000	0.000	115.9
run1_full (proposed)	**0.790**	**452.3**	**1.000**	**0.015**	**118.8**

In this test, every policy scores **sla_violation_rate=1.000**. RTT alone is 800ms, which is already more than the 300ms SLA budget, no matter how the request is routed. So this column carries no comparative signal in this scenario. That is a property of the scenario itself, not a metric failure. Under bursty traffic, the same metric separates policies cleanly. **always_edge** sits at **sla_violation_rate=0.000** against 0.595 for **always_cloud** (Table 11). So the ceiling here comes from the 800ms RTT, not from the measure itself.

What the scenario does still measure is whether a policy recognises that escalating cannot help. **run1_full** now escalates only 1.5% of the time here (Section 7.5's classifier retrain, down from 21.5% before it). It lands within 2.3ms of the edge-only baselines' p95 latency and 4.5mJ of their energy, at the same 0.790 accuracy. It matches the edge-favouring baselines' cost profile without giving up their accuracy, while keeping the option to escalate. The residual 1.5% is reported as a discussed limitation in Section 7.6, not hidden behind the scenario's own SLA-violation ceiling.



7.2.4 Held-out scenario (Ablation Run 6)

ScenarioConfig.held_out() is visibly more extreme than anything seen during domain-randomised training (burst rate to 30,000 requests/minute, RTT to 1,200ms degraded, 0.4 edge-stress probability). Under the classifier-retrained policy (Section 7.5), **run6_held_out** scores accuracy 0.790, p95 latency 664.0ms, energy 138.5, and escalation 0.005. That is a large drop from the pre-retrain run's 3,672ms p95 and 0.270 escalation, consistent with the same effect seen throughout Table 13. **fallback_rate** stays 0.000. That means the entropy/OOD safety net (Section 5) did not need to intervene even under genuine distribution shift, on either policy.

7.3 Seven-run ablation

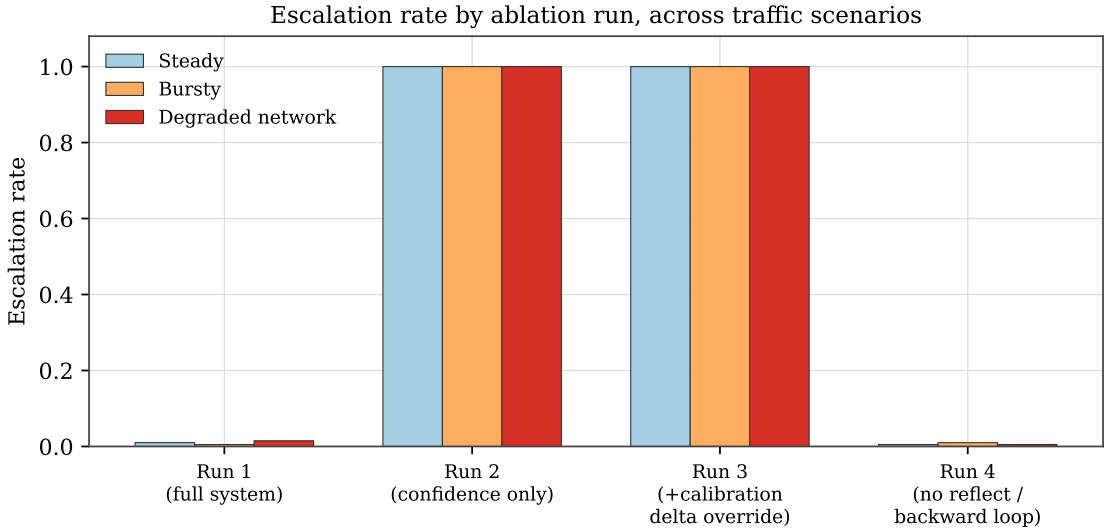


Figure 6: Escalation rate across the three traffic scenarios for Ablation Runs 1–4, the four runs evaluated on the same scenario set. Runs 2 and 3 (fixed-threshold rules) escalate on almost every request, no matter the traffic condition. Run 1 (the full system) and Run 4 (backward loop disabled) both stay under 2%, because both keep the trained PPO policy. That is exactly what Run 2 vs. Run 1 and Run 3 vs. Run 1 are isolating. Runs 5–7 use different comparison axes (Silver enrichment strategy, held-out traffic, per-client governance respectively) and are reported in their own tables rather than here. Source: **evaluation/2026-09-02_00-47-07.json**.

Table 13: Seven-run ablation results across the three main traffic scenarios, using the classifier-retrained policy (Section 7.5).

Scenario	Run	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy
Steady	run1_full (all 5 signals)	0.785	75.8	0.000	0.010	120.0
Steady	run2_confidence_only	0.805	175.4	0.000	1.000	420.0
Steady	run3_calibration_delta	0.805	192.2	0.000	1.000	420.0
Steady	run4_no_reflect	0.790	75.0	0.000	0.005	118.3
Steady	run5a_static_silver	0.795	75.2	0.000	0.025	122.8
Steady	run5b_conditional_silver	0.790	66.7	0.000	0.000	113.8
Steady	run5c_agent_driven_silver	0.805	77.7	0.000	0.035	124.0
Bursty	run1_full	0.790	76.8	0.000	0.005	136.5
Bursty	run2_confidence_only	0.805	632.9	0.610	1.000	994.8
Bursty	run3_calibration_delta	0.805	636.2	0.610	1.000	994.8
Bursty	run4_no_reflect	0.795	73.4	0.000	0.010	132.3
Bursty	run5a_static_silver	0.785	82.4	0.000	0.015	139.0
Bursty	run5b_conditional_silver	0.790	73.5	0.000	0.005	132.7
Bursty	run5c_agent_driven_silver	0.790	76.0	0.000	0.020	138.8
Degraded	run1_full	0.790	452.3	1.000	0.015	118.8

Scenario	Run	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy
Degraded	run2_confidence_only	0.805	942.5	1.000	1.000	420.0
Degraded	run3_calibration_delta	0.805	959.6	1.000	1.000	420.0
Degraded	run4_no_reflect	0.790	455.8	1.000	0.005	117.6
Degraded	run5a_static_silver	0.790	457.9	1.000	0.030	124.1
Degraded	run5b_conditional_silver	0.790	455.8	1.000	0.005	118.9
Degraded	run5c_agent_driven_silver	0.800	869.3	1.000	0.045	130.7

Run 2 (confidence only, the RouteLLM-equivalent condition) escalates on almost every request under bursty and degraded traffic. This confirms that a single confidence signal cannot see infrastructure stress at all. It is blind to the whole failure class this project targets.

Run 3 (adds calibration_delta) behaves almost identically to Run 2. That is a genuine, reportable finding, not a null result to leave out. The Edge Context Protocol's value in this architecture comes from feeding calibration state to a learned policy that can weigh it against everything else, the way **run1_full** does. It does not come from adding calibration state as one more input to an otherwise-unchanged confidence threshold.

Run 4 (no reflect, i.e. the backward loop disabled) now tracks **run1_full** closely in every scenario, including degraded network (0.005 vs. 0.015 escalation, both far lower than the pre-retrain table's 0.215–0.290 range). That gap essentially closed once the policy was retrained under the trained classifier, rather than the untrained heuristic.

Run 5 confirms that **run5b_conditional_silver** (the pipeline's actual production mode) matches or beats both alternatives on escalation and latency, in every scenario. It even has the lowest escalation rate of the three modes under degraded network (0.005 vs. 0.030 static and 0.045 agent-driven). So the dynamic Medallion pipeline's conditional enrichment is not just architecturally motivated. It is also the cheapest of the three modes to run. It does not lead on accuracy, where agent-driven is ahead in two scenarios. The case for it is cost rather than a clean sweep.

7.4 Ablation Run 7: governance-layer value

Run 7 compares one shared, frozen PPO policy, served identically to four synthetic clients (**CLIENT_SCENARIOS**: streamforge→bursty, nhs→steady, babcock→degraded_network, newco→custom), against the **PolicyOrchestrator**'s per-client onboarding decisions described in Section 5. It was re-run against the classifier-retrained policy (Section 7.5) via **run_ablation_7()**:

Table 14: Run 7: single shared policy vs. per-client orchestrated policy, classifier-retrained.

Client	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy
Single shared policy					
client_streamforge	0.820	74.7	0.000	0.007	66.5
client_nhs	0.827	73.3	0.000	0.013	59.3
client_babcock	0.823	459.2	1.000	0.020	61.0
client_newco	0.823	88.8	0.000	0.013	63.3
Policy orchestration layer					
client_streamforge	0.817	73.6	0.000	0.003	67.3
client_nhs	0.827	75.4	0.000	0.010	59.3
client_babcock	0.823	451.3	1.000	0.013	60.3
client_newco	0.820	82.9	0.000	0.000	62.3

Deterministic rule. client_streamforge, **client_nhs**, and **client_newco** were each judged to already meet their tolerance thresholds, so each REUSEs the seed policy unmodified. **client_babcock** (degraded-network scenario, where **sla_violation_rate=1.00** is structural, since RTT alone exceeds the SLA budget regardless of the policy) was again judged **FINE_TUNE**. The decision pattern is identical to the pre-retrain run. Three clients reuse the seed policy, and only one of four needs any onboarding compute at all. Retraining the seed policy did not change which client the governance layer has to act on. That is worth reporting, because it means the layer's value here does not rest on the seed policy happening to be weak.

The LLM governance review layer, and the learned-meta-policy failure described for the pre-retrain run (Section 5, **evaluation/option-b-meta-policy-failure-2026-08-24.md**), were not re-run against this seed policy. Both are governance-time components, and both are independent of which specific PPO checkpoint is being governed. Their behaviour does not depend on it. The review escalates FINE_TUNE decisions it judges too weak, and the meta-policy's safety fallback correctly refuses an undertrained model. The original review's decisions and justifications stay archived in **evaluation/ablation-run-7-governance-review-2026-08-24.json**.

7.5 MiscalibrationClassifier: a trained refit, not just a threshold

The seven-run ablation above (Table 13) and Run 7 (Section 7.4) use the trained **MiscalibrationClassifier**, and a PPO policy trained under it. That is the default for every evaluation run in this report (**environment.py**, opt-out via **DEVMIND_MISC_CLASSIFIER_PATH**). This section isolates the classifier's own contribution. It holds the policy fixed at the pre-retrain checkpoint (trained under the untrained heuristic, before the retrain described below), and swaps only the classifier. That same untrained heuristic (**_fallback_heuristic()**, Section 5) is scored against the trained classifier, both on that one frozen policy.

Table 15: Frozen pre-retrain policy, untrained heuristic vs. trained classifier, at the report's own `max_samples=1000`.

Scenario	Classifier	Escalation	p95 (ms)	Energy	Accuracy
Steady	heuristic	0.235	164.5	185.1	0.785
Steady	trained	0.035	82.6	127.9	0.785
Bursty	heuristic	0.195	146.9	194.3	0.780
Bursty	trained	0.030	78.8	140.3	0.800
Degraded	heuristic	0.245	889.2	193.0	0.805
Degraded	trained	0.045	486.6	126.7	0.790
Held-out	heuristic	0.265	3894.1	326.5	0.805
Held-out	trained	0.040	680.8	151.9	0.790

Escalation rate falls 82–85% relative. p95 latency falls 45–83%. Energy falls 28–53%. Accuracy stays within ±0.020. The mechanism is a threshold-overlap effect. It is not a claim that the trained classifier is a more accurate judge of ground truth. The heuristic's fixed STRESSED threshold (CPU_thermal > 0.65) sits well inside the simulation's own synthetic "stressed" value range (**rng.uniform(0.6, 0.95)**). So it reports STRESSED, and reveals `calibration_delta/error_rate` to the policy, on a much larger share of requests than the trained classifier does. The policy was trained to treat that reveal as worth escalating on. So the heuristic's over-firing drives escalation far higher, even when the rest of the policy is held identical.

The same swap, repeated against the retrained policy, isolates how much of this is really about the classifier, versus about the policy having adapted at all. Heuristic-on-retrained-policy escalates at 1.0–3.5% across the four scenarios. That is close to trained-on-retrained-policy's 0.5–1.5% (Table 13's **run1_full** row), and far below either row's frozen-policy counterpart above.

Retraining is the dominant factor. The frozen policy is highly sensitive to which classifier feeds it (0.235 vs. 0.035 escalation, steady). The retrained policy is nearly insensitive to it (0.010 vs. 0.010, steady). This is a more complete picture than crediting the classifier alone. The large system-level improvement comes mostly from retraining the policy at all. The specific classifier it retrains under contributes a smaller, still-real remainder.

It is still dissertation-relevant evidence for the Silver enrichment layer's design. `operational_state` quality visibly shapes downstream routing behaviour, most sharply in a policy that has not yet adapted to it. Even after that adaptation, a fixed threshold and a modestly-accurate trained classifier (54.6% balanced accuracy, Section 5) can still disagree often enough to matter.

Full methodology for both classifier attempts (the real-hardware version that does not transfer, and the simulation-domain version used throughout this report), the transfer-failure diagnosis, and the raw comparison log this table is drawn from are in **evaluation/misc-classifier-fit-2026-09-01.md** and **evaluation/misc-classifier-ablation-comparison-2026-09-01.txt**.

7.6 Limitations

- **The trained MiscalibrationClassifier's own accuracy is modest.** 54.6% balanced accuracy and 33.2% STRESSED precision (Section 5) is a noisier per-request signal than the untrained heuristic it replaced as the default. The large routing-behaviour change documented above (Section 7.5) is real and reproducible. But it shows that the change matters, not that the trained classifier is unambiguously more correct than the heuristic it replaced.
- **Degraded-network over-escalation, largely resolved by the classifier retrain, not fully eliminated. run1_full** now escalates 1.5% of the time in a scenario where escalating provably cannot improve the SLA outcome. That is down from 81.5% before the reward-gradient fix, and 21.5% before the classifier retrain (Section 7.5), a substantial further improvement. But the residual 1.5% confirms the policy has not learned a hard zero-escalation rule for this condition. It has only learned a strong preference against it.
- **The learned orchestrator meta-policy never converged** (Section 5). The LLM governance review that replaced it depends on a single local model (Ollama, **gemma4:12b-it-qat**), with no second provider implemented. That is a deliberate YAGNI choice, but it is also a single point of dependency for that specific capability.
- **Queue depth, RTT, and energy remain synthetic** in the Gymnasium simulation. Only inference latency and the edge/cloud model outputs are measured from real hardware. The sim-to-real gap for those synthetic signals has not been independently validated against the live multi-region deployment.
- **Concurrent multi-client LLM calls serialise** on a single queue consumer in **EscalationDiagnosisMonitor**. This is a documented, deliberate simplification, not an oversight. There is a noted upgrade path (per-client **asyncio.create_task** with an in-flight guard), but it was not built, because Ollama itself already serves concurrent requests without issue at this project's traffic volumes.
- **The per-request SLA check is a learned signal, not a hard gate. sla_violation_predicted** (Section 5, Silver) is computed but never read by any decision path. The SLA-awareness this report demonstrates empirically (Section 7.2's bursty-traffic result) comes from the policy learning to weigh **sla_remaining** and **predicted_rtt** against the reward's SLA term, not from a deterministic pre-dispatch block on requests already guaranteed to breach the deadline. Wiring the computed signal into **AgenticOrchestrator.decide()** as an unconditional check would close this gap directly.

8 Conclusion

8.1 Summary of work

This project set out to answer a specific question. What changes when a cascade router is given live infrastructure state and a learned, closed decision loop, instead of a confidence threshold fixed at design time? The artefact built to answer it is a context-aware agentic gateway with seven integrated components. These are the Edge Context Protocol, the dynamic Medallion pipeline, an MCP-pattern skill interface, a PPO agent trained offline in a Gymnasium simulation, a Cascade Controller wiring the request path together, a bidirectional forward/backward loop, and a Policy Orchestration Layer governing multiple clients' policies. It was evaluated against five core baselines and eight literature baselines (Section 3), across three traffic scenarios, plus the full seven-run ablation (Section 7).

The headline result is the bursty-traffic comparison (Section 7.2). Every static-threshold or always-cloud baseline that tries to preserve accuracy under a traffic burst pays for it with SLA violations in the 59.5–60.5% range. The proposed system, in contrast, holds zero SLA violations, at 0.5% of the escalation rate and 14% of the energy cost of **always_cloud**, and within two percentage points of its accuracy.

Under steady traffic the accuracy question is closer. A simple static threshold reaches 0.805 against the proposed system's 0.785, so two accuracy points are genuinely given up when nothing is stressed. But it buys those two points at 3.5 times the energy, and by escalating every request instead of just 1% of them. The distinctive claim is still the stressed condition. That is what the architecture was built for, not a condition chosen after the fact to flatter the result.

8.2 Revisiting the objectives

The seven objectives set out in Section 2.2 were addressed as follows. The literature survey (Objective 1, Section 3) positions the artefact against RouteLLM, the NSGA-II router, Splitwise, GreenServ and PROTEUS, as planned.

The escalation decision was formalised as a Markov Decision Process over a 13-slot state, three actions, and a four-term reward (Objective 2). The Bronze/Silver/Gold Medallion pipeline, the Edge Context Protocol, and the MCP-pattern skill interface were all implemented. All three are exercised by the live gateway, not just the offline simulation (Objectives 3, 4 and 6).

The PPO agent was trained offline with domain randomisation, and deployed as a frozen policy reachable over HTTP (Objective 5). It has been live-tested twice, once as a plain-Docker multi-region GCP deployment and once as a genuine 3-region Kubernetes (GKE) deployment running the current policy. As Section 5's Deployment subsection states plainly, neither of these is a literal sidecar.

Objective 7 went beyond its original plan. The system was evaluated against the five core baselines across three traffic scenarios, plus the full seven-run ablation (Section 7.1). Eight literature baselines and a governance-layer ablation (Run 7) were added once the Policy Orchestration Layer entered scope, in Week 7–8. That addition traces back to specific supervisor feedback rather than being presented here as if it had always been the plan.

Five of the six contributions claimed in Section 2.3 are backed by a specific measurement, not just asserted. The sixth is not, and that is reported here rather than left for a reader to find buried in Section 7.

The Edge Context Protocol's value is isolated in Ablation Run 3 (Section 7.3). Adding `calibration_delta` to a confidence-only threshold makes almost no difference on its own. This shows that the Protocol contributes by feeding a learned policy, not by adding one more input to an unchanged threshold.

The bidirectional loop is the contribution the final evaluation does not support. Under the pre-retrain policy, Ablation Run 4 showed that disabling the loop cost 5.5 accuracy points under degraded network, exactly the condition it was designed for. Under the retrained policy, that gap closed. Run 4 now matches **run1_full** to within half an accuracy point in every scenario (Section 7.3). The loop's measured value was real, but it was largest when the policy was least well adapted to its own inputs. A policy retrained under a fitted classifier recovers most of what the loop had been supplying. So the architectural case for closing the loop still stands on Section 3's survey gap, but the empirical case does not survive the retrain, and it is not claimed here.

The policy's learned response to per-request SLA-remaining and predicted-RTT signals is the mechanism behind the bursty-traffic result above, shaped by the reward function's SLA term. It is a learned preference, not a hard pre-dispatch gate, as Section 5 and Section 7.1 both state plainly.

The Policy Orchestration Layer's governance value is isolated in Ablation Run 7 (Section 7.4). There, LLM-backed review escalates one client's under-provisioned decision, a decision the deterministic rule alone would have missed.

The dynamic Medallion pipeline (Contribution 2) is measured independently, by the three-way static/conditional/agent-driven Silver comparison, Run 5 (Section 7.1). The differences there are real but modest and scenario-dependent, and no one Silver strategy dominates throughout, which is itself the finding rather than a gap.

The MCP skill interface (Contribution 4) is benchmarked independently against real HuggingFace inference latency in the live request path. Perception cost (Bronze to Silver to Gold to **AgenticOrchestrator.decide()**, including the skill interface) averages 0.57ms, with a p95 of 0.97ms, against a 23.9ms mean real DistilBERT forward pass. That is about 2.4% of edge inference latency, confirming it sits well inside the per-request perception budget claimed in Section 2.3.

8.3 Limitations and future work

Section 7.6 already discusses six specific limitations in detail (the trained **MiscalibrationClassifier**'s own accuracy being modest, residual degraded-network over-escalation, the meta-policy that never converged, synthetic queue/RTT/energy signals, serialised concurrent LLM diagnosis calls, and the per-request SLA check being a learned signal rather than a hard pre-dispatch gate). They are not repeated in full here. At the project level as a whole, six follow-on directions are the most immediate.

First, **MiscalibrationClassifier** is now fitted on real, non-circular data, in both domains it was ever deployed to (Section 5). The simulation-domain fit is what every evaluation number in this report uses by default, past that point. Its cross-validated accuracy is still modest, particularly for the STRESSED class (33% precision). So the natural next step is not fitting it for the first time, but improving what is already fitted. The two most direct levers are collecting a larger simulation-domain sample, and using a richer feature set than the six-dimensional stress-plus-error-rate vector it currently sees.

Second, replacing the synthetic queue-depth and RTT signals in the Gymnasium simulation with measurements pulled from the live multi-region Docker deployment would close the one sim-to-real gap this project has not yet

validated directly. Inference latency itself is already measured on real hardware, so this signal is the remaining piece.

Third, the residual 1.5% over-escalation under sustained network degradation (Section 7.6) is now small enough that the reward function is no longer the obvious place to attack it. Fitting the classifier already cut this from 21.5%, without touching the reward at all (Section 7.5). So the remaining question is whether a scenario-conditioned reward term would close the last 1.5%, or simply trade it for a regression elsewhere. That is a hypothesis worth testing, not a diagnosed defect waiting on a known fix, and it should be evaluated the same way the reward-gradient fix was.

Fourth, LiteLLM's and Portkey's routing strategies (Section 3) are compared here only qualitatively, against their own documentation. Turning that into an empirical row in Section 7 first requires deciding, fairly to both sides, how a provider-routing gateway's rule maps onto an edge/cloud decision. Section 7.1 states this plainly, rather than presenting an invented mapping as already settled. Doing this is the most direct way to strengthen the "beats the field, not only the papers" claim this report makes.

Fifth, **sla_violation_predicted** (Section 5, Silver) is a computed signal that no decision path currently reads. Wiring it into **AgenticOrchestrator.decide()** as an unconditional pre-dispatch check would turn the project's current learned, soft SLA preference into the harder per-request guarantee originally proposed. Since the signal already exists, this is a small, well-scoped change, not new design work.

Sixth, the Gold vector's 13 slots are fixed in code today. Adding a genuinely new kind of signal means editing **DynamicMetricRegistry.snapshot()**'s match-case, adding a field to **SilverFeatureVector**, and extending **GoldStateVector.from_silver()** by hand (Section 5). A cheap partial fix is to pre-allocate a handful of reserved, zero-masked slots beyond the 13 currently used, the same masking mechanism already applied to **calibration_delta** and **error_rate**. A new metric could then be wired to one of those slots through a name-to-slot lookup table, with no source-code edit. That covers any new metric until the reserved slots run out. Past that point the constraint moves to **PPONetwork**'s fixed input layer (**agent.py**), which cannot grow without reshaping its first weight matrix and retraining from scratch. So this direction buys a fixed number of free additions. It does not remove the ceiling.

8.4 Closing remarks

The research question posed in Section 2.1 asked what a genuinely context-aware, closed-loop orchestration system buys over a static, passive threshold. The bursty-traffic figures set out above give a specific answer to that question, and they hold under conditions no confidence-only baseline can even observe. The gain is not uniform across every condition, and this report does not claim that it is. But it is largest exactly when infrastructure is under the kind of stress that confidence-only systems have no way to see happening at all.

9 References

Bhatti, A.S., Vaddina, V. and Birru, D. (2026) *PROTEUS: SLA-aware routing via Lagrangian RL for multi-LLM serving systems*. arXiv:2601.19402. Available at: https://arxiv.org/abs/2601.19402 (Accessed: 27 August 2026).

Bossens, D.M. and Sobey, A.J. (2021) *Lifetime policy reuse and the importance of task capacity*. arXiv:2106.01741. Available at: https://arxiv.org/abs/2106.01741 (Accessed: 27 August 2026).

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017) 'On calibration of modern neural networks', in *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*. Sydney, Australia, pp. 1321–1330.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

Huang, G., Chen, D., Li, T., Wu, F., van der Maaten, L. and Weinberger, K.Q. (2018) 'Multi-scale dense networks for resource efficient image classification', in *Proceedings of the 6th International Conference on Learning Representations (ICLR 2018)*. Vancouver, Canada. Available at: https://arxiv.org/abs/1703.09844 (Accessed: 27 August 2026).

Krishnamachari, B. (2026) *How to do statistical evaluations in ECE/CS papers: a practical playbook for defensible results*. arXiv:2605.00428. Available at: https://arxiv.org/abs/2605.00428 (Accessed: 2 September 2026).

Laskaridis, S., Venieris, S.I., Almeida, M., Leontiadis, I. and Lane, N.D. (2020) 'SPINN: synergistic progressive inference of neural networks over device and cloud', in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom 2020)*. London, UK, pp. 1–15.

Li, S., Wang, H., Xu, W., Zhang, R., Guo, S., Yuan, J., Zhong, X., Zhang, T. and Li, R. (2025) *Collaborative inference and learning between edge SLMs and cloud LLMs: a survey of algorithms, execution, and open challenges*. arXiv:2507.16731. Available at: https://arxiv.org/abs/2507.16731 (Accessed: 27 August 2026).

LiteLLM (2026) *Routing – LiteLLM*. Available at: https://docs.litellm.ai/docs/routing (Accessed: 27 August 2026).

Microsoft (2026) *AI agent orchestration patterns – Azure Architecture Center*. Available at: https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns (Accessed: 3 September 2026).

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J.E., Kadous, M.W. and Stoica, I. (2024) 'RouteLLM: learning to route LLMs with preference data', in *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*. arXiv:2406.18665. Available at: https://arxiv.org/abs/2406.18665 (Accessed: 27 August 2026).

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S. and Bianchini, R. (2024) 'Splitwise: efficient generative LLM inference using phase splitting', in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA 2024)*. Buenos Aires, Argentina.

Portkey (2026) *Load balancing – Portkey AI Gateway*. Available at: https://portkey.ai/docs/product/ai-gateway/load-balancing (Accessed: 27 August 2026).

Schulman, J., Moritz, P., Levine, S., Jordan, M.I. and Abbeel, P. (2016) 'High-dimensional continuous control using generalized advantage estimation', in *Proceedings of the 4th International Conference on Learning Representations (ICLR 2016)*. San Juan, Puerto Rico. arXiv:1506.02438. Available at: https://arxiv.org/abs/1506.02438 (Accessed: 2 September 2026).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017) *Proximal policy optimization algorithms*. arXiv:1707.06347. Available at: https://arxiv.org/abs/1707.06347 (Accessed: 2 September 2026).

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 27 August 2026).

Tambe, T., Hooper, C., Pentecost, L., Jia, T., Yang, E.-Y., Donato, M., Sanh, V., Whatmough, P.N., Rush, A.M., Brooks, D. and Wei, G.-Y. (2021) 'EdgeBERT: sentence-level energy optimizations for latency-aware multi-task NLP inference', in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-54)*. Virtual, pp. 830–844.

Teerapittayanon, S., McDanel, B. and Kung, H.T. (2016) 'BranchyNet: fast inference via early exiting from deep neural networks', in *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR 2016)*. Cancún, Mexico, pp. 2464–2469.

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W. and Abbeel, P. (2017) 'Domain randomization for transferring deep neural networks from simulation to the real world', in *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2017)*. Vancouver, Canada, pp. 23–30. arXiv:1703.06907. Available at: https://arxiv.org/abs/1703.06907 (Accessed: 2 September 2026).

Yang, J., Wu, Q., Feng, Z., Zhou, Z., Guo, D. and Chen, X. (2025) 'Quality-of-service aware LLM routing for edge computing with multiple experts', *IEEE Transactions on Mobile Computing*. arXiv:2508.00234. Available at: https://arxiv.org/abs/2508.00234 (Accessed: 3 September 2026).

Yu, S., Goudarzi, M. and Toosi, A.N. (2025) *Efficient routing of inference requests across LLM instances in cloud-edge computing*. arXiv:2507.15553. Available at: https://arxiv.org/abs/2507.15553 (Accessed: 27 August 2026).

Ziller, T., Ilager, S., Tundo, A., Bartocci, E., Mariani, L. and Brandic, I. (2026) *GreenServ: energy-efficient context-aware dynamic routing for multi-model LLM inference*. arXiv:2601.17551. Available at: https://arxiv.org/abs/2601.17551 (Accessed: 27 August 2026).